MIP trešā moduļa apkopojums otrajam testam

3.1. tēma: Heiristiski informēta pārmeklēšana un stāvokļu telpas grafa sarežģītība

Tēmas mērķis ir definēt heiristiskā novērtējuma funkciju un attīstīt prasmes to izstrādāt konkrētajam problēmsfēras uzdevumam.

Sasniedzamie studiju rezultāti - Pēc šīs tēmas apgūšanas studenti zinās:

- zinās, kas ir heiristiskas zināšanas cilvēku dzīvē un stāvokļu telpas grafa pārmeklēšanā;
- zinās, kas ir heiristiskā novērtējuma funkcija;
- zinās, kādi ir heiristiski informētas pārmeklēšanas algoritmu darbības soļi, ieejas un izejas;
- zinās, kā tiek mērīta virsotņu kvalitāte stāvokļu telpas grafā;
- zinās, kāpēc ir jāmeklē skaitļošanas izmaksu kompromiss, kad tiek lietota heiristiskā novērtējuma funkcija;
- pratīs definēt heiristiskā novērtējuma funkciju;
- pratīs noteikt stāvokļu telpas grafa sarežģītību;
- zinās metodes stāvokļu telpas grafa sarežģītības samazināšanai.

Līdz šim iepriekšējās tēmās runājot par intelektiskām datorsistēmām, kas spēj risināt problēmas, esam apskatījuši tikai neinformētas pārmeklēšanas algoritmus, kuri stāvokļu telpas grafā nespēj atšķirt ceļus, kas ļauj sasniegt pārmeklēšanas mērķa virsotni, no ceļiem, kas to neļauj izdarīt, un būtībā secīgi pēta visus pieejamus ceļus līdz brīdim, kad sameklē tādu ceļu, kas ved uz pārmeklēšanas mērķa virsotni un tad izdod to kā atrisinājuma ceļu. Šādu algoritmu izmantošana praksē dažreiz var prasīt ievērojamus skaitļošanas resursus. Pārmeklēšanas mērķa virsotnes atrašanas efektivitāti var uzlabot, izmantojot heiristiski informētas pārmeklēšanas algoritmus. Taču pirms apskatīt specifiskus algoritmus, šajā tēmā noskaidrosim, kas ir heiristiskas zināšanas cilvēku dzīvē un kāpēc tās tiek lietotas, kā arī kā heiristiskas zināšanas var tikt atspoguļotas un izmantotas intelektiskās datorsistēmās.

Mūsdienās plaši izmantojamais vārdu salikums “life hack” atbilst tam, ko saprot ar heiristiskām zināšanām. Vispārīgā gadījumā **heiristiskas zināšanas** ir pieredzē-balstītie likumi, kas ļauj cilvēkiem efektīvāk risināt problēmas, pieņemt lēmumus un spriest. Tā kā problēmai var būt liels skaits veidu, kā to atrisināt, cilvēki izmanto stratēģijas (heiristiskas metodes), lai efektīvā veidā nonāktu no problēmrisināšanas sākumstāvokļa uz problēmrisināšanas mērķa stāvokli (Eysenck & Keane, 2000). Heiristiskas zināšanas var tikt uzskatītas par īkšķa likumiem, kas nedod perfektu rezultātu, bet parasti ar daudz mazāku piepūli noved pie laba vai vismaz pieņemama rezultāta (Lieberman, 2012). Kognitīvajā psiholoģijā heiristiskas zināšanās sakņotas problēmrisināšanas metodes bieži tiek pretstatītas tādām metodēm, kas garantē precīza problēmas atrisinājuma

iegūšanu, ja tāds eksistē (Eysenck & Kean, 2000; Goldstein, 2008). Goldšteins (2008) dod šādu piemēru: iedomājaties, ka jums ir jāatrod, kur jūsu mājā ir paslēpies kaķis. Jūs varētu ļoti rūpīgi apskatīt visas istabas vienu aiz otras un galu galā atrast kaķi, kaut gan, iespējams, patērējot tam daudz laika. Cita pieeja, kas izmanto heiristiskas zināšanas, ir veikt meklēšanu tikai tajās vietās, kurās kaķis parasti slēpjas. Tas ne vienmēr ļaus jums atrast kaķi, bet, ja tomēr tas sanāks, tad tas būs ātrāk, nekā apskatot visas istabas pēc kārtas. Praksē bieži vien labam risinājumam, kas tiek atrasts ātrāk, tiek dota priekšroka salīdzinājumā ar risinājumu, kas ir optimāls, bet ir dārgs resursu patēriņa ziņā (Ertel, 2011). Augstskolas studentiem ātri veidojas heiristiskas zināšanas par to, kādas lekcijas ir vērts apmeklēt klātienē un kādas nav vērts. Viens no izplatītākajiem heiristisku zināšanu izmantošanas piemēriem sistēmu administratoru darbā ir ieteikums lietotājam restartēt datoru, ja tam ir kļūme.

Stāvokļu telpas grafā heiristikas zināšanas ir likumi, kas ļauj izvēlēties tos ceļus stāvokļu telpas grafā, kuri visticamāk ved uz pieņemamu problēmas risinājumu (Luger, 2009). Zemāk dota tabula atspoguļo, kas tiek saprasts ar heiristiskām zināšanām cilvēku dzīvē un kā tās tiek atspoguļotas un izmantotas intelektiskās datorsistēmās. Heiristiskas zināšanas tipiski lieto šādu problēmu risināšanai datorzinātnē: spēļu izveide, teorēmu pierādīšana, medicīnas diagnostikas sistēmas, datorredzes sistēmas, finanšu sistēmas, bioinformātika, dabīgās valodas sistēmas un antivīrusu programmatūra.

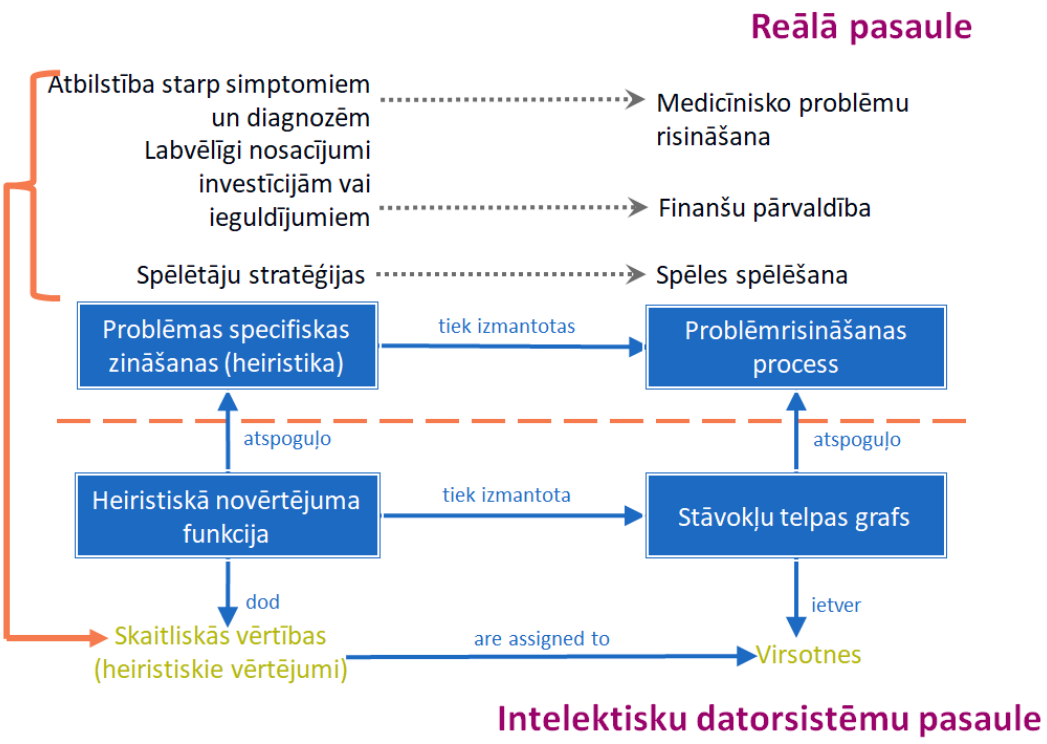
1. tabula. Heiristiskas zināšanas

Kritērijs	**Cilvēki**	**Intelektiskas datorsistēmas**
Kas tās ir?	Likumi, kas atbalsta efektīvāku problēmrisināšanu	Heiristiskā novērtējuma funkcija un tās vērtības
Kā tās tiek iegūtas?	Lielākoties tās tiek iegūtas pieredzes ceļā, bet tās var arī iegūt, izmantojot intuīciju, novērojumus vai mācīšanos darot	Šo funkciju izstrādā sistēmas izstrādātāji ciešā sadarbībā ar problēmsfēras ekspertiem, pielietojot arī mašīnmācīšanās un optimizācijas metodes
Ko tās veicina?	Tās veicina efektīvāku problēmrisināšanu, lēmumu pieņemšanu un spriešanu	Funkcijas izmantošana veicina perspektīvāku ceļu (tādu, kas ved uz pārmeklēšanas mērķa virsotni) izvēli pārmeklēšanas procesā
Kādu ieguvumu tās dod?	Šo zināšanu lietošana samazina cilvēku kognitīvu slodzi, kura būtu vajadzīga problēmas dziļai analīzei	Funkcijas lietošana samazina pārmeklējamo virsotņu skaitu un līdz ar to arī kopējo pārmeklēšanas laiku
Kad tās tiek izmantotas?	Tās tiek izmantotas visbiežāk sarežģītām problēmām ar nepilnu informāciju, kad nav iespējams problēmu aprakstīt vai novērtēt pilnībā, bet tajā pašā laikā cilvēkiem ir daudz heiristisku zināšanu saistībā ar ikdienas situācijām	Tās tiek izmantotas, lai risinātu a) problēmas, kurām nav precīzi zināms atrisinājums, jo pastāv zināma nenoteiktība pieejamos datos vai problēmas formulējumā vai b) problēmas, kurām ir precīzs risinājums, bet nav pietiekami daudz skaitļošanas resursu, lai to atrastu

Kritērijs	Cilvēki	Intelektiskas datorsistēmas
Kādi trūkumi piemīt?	Tās var būt maldīgas, neļaujot atrisināt konkrētu problēmu, jo katram cilvēkam ir atšķirīga dzīves pieredze un heiristiskas zināšanās balstītie likumi ir izteikti individuāli	Funkcijas izmantošana var novest pie vislabākā risinājuma, bet var arī novest tikai pie pieļaujama risinājuma, vai neatrast atrisinājumu vispār

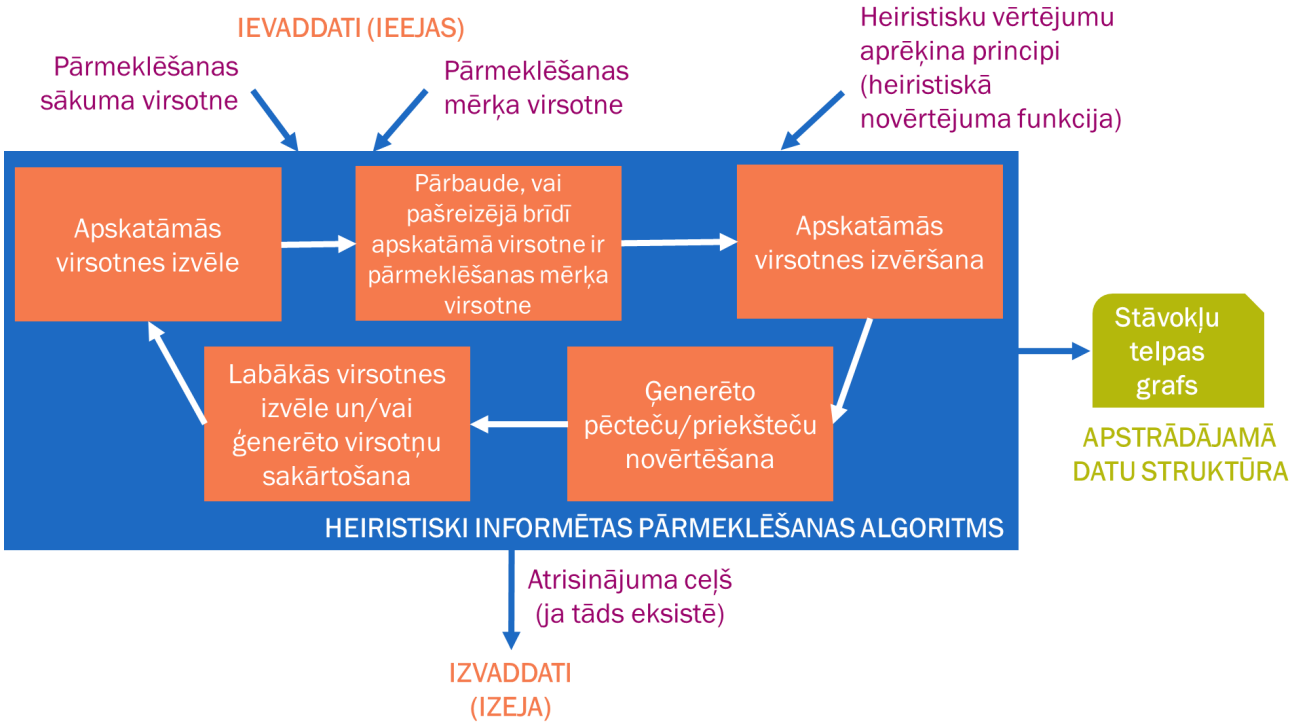
Tādējādi 1.tabulā ir definēts, ka heiristiskas zināšanas intelektiskās datorsistēmās tiek atspoguļotas **heiristiskā novērtējuma funkcijas** veidā. Šī funkcija palīdz pārmeklēšanas algoritmam noteikt virzienu, kurā atrodas pārmeklēšanas mērķa virsotne (Poole, 2010). Funkcijas mērķis ir atrast problēmas risinājumu ar minimālām izmaksām (Ertel, 2011).

Heiristiskā novērtējuma funkcijas ideju paskaidro 1.attēls. Reālajā pasaulē cilvēki risina dažādu veidu problēmas, piemēram, spēlē spēlēs, veic finanšu resursu pārvaldību, risina ar veselību saistītos jautājumus un citas. Šim nolūkam cilvēki lieto problēmai specifiskas zināšanas, ko sauc par heiristiskām zināšanām, piemēram, spēlēs tās ir spēlētāju stratēģijas, finanšu pārvaldībā - labvēlīgi nosacījumi investīcijām vai ieguldījumiem, medicīnā - atbilstības likumi starp simptomiem un diagnozēm. Intelektisku datorsistēmu pasaulē problēmrisināšanas process tiek atspoguļots stāvokļu telpas grafā, kas sevī ietver problēmrisināšanas procesa stāvokļiem atbilstošas virsotnes. Savukārt heiristiskā novērtējuma funkcija atspoguļo problēmai specifiskas zināšanas un tā tiek izmantota stāvokļu telpas grafā. Funkcija dod skaitliskās vērtības, kuras sauc par heiristiskiem vērtējumiem. Tādējādi problēmai specifiskas zināšanas tiek izteiktas matemātiskā veidā. Skaitliskās vērtības tiek piešķirtas stāvokļu telpas grafa virsotnēm un izmantotas, lai efektīvāk meklētu pārmeklēšanas mērķa virsotni.



1. att. Heiristisku zināšanu izmantošana cilvēku dzīvē un intelektisku datorsistēmu izstrādē

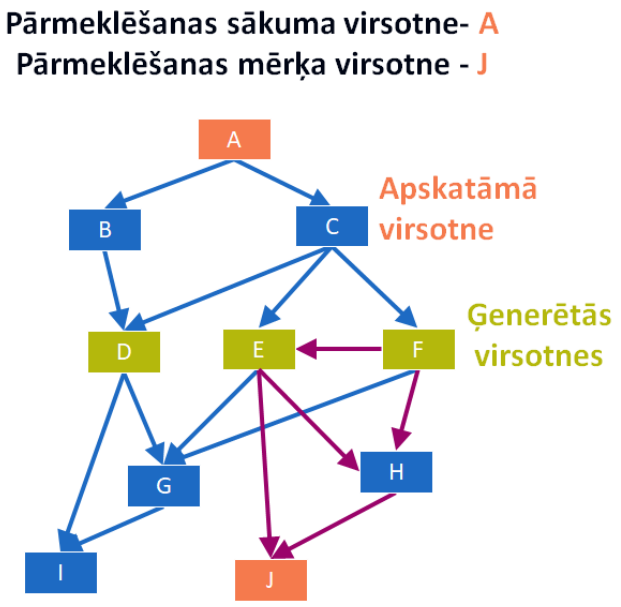
Vispārīgā gadījumā atšķiras veids, kā neinformētas pārmeklēšanas un heiristiski informētas pārmeklēšanas algoritmi meklē atrisinājuma ceļu (2.att.). Abu kategoriju algoritmi izmanto pārmeklēšanas sākuma virsotni un pārmeklēšanas mērķa virsotni kā ieejas, izdod atrisinājuma ceļu kā rezultātu un izmanto stāvokļu telpas grafu, lai to meklētu. Turklāt abu kategoriju algoritmiem ir trīs identiski darbības soļi: 1) izvēlēties apskatāmo virsotni, 2) pārbaudīt, vai apskatāmā virsotne ir pārmeklēšanas mērķa virsotne, un 3) izvērst apskatāmo virsotni, ja tā nav pārmeklēšanas mērķa virsotne. Taču heiristiski informētas pārmeklēšanas algoritmiem papildus ir nepieciešami principi, kā rēķināt virsotņu **heiristiskus vērtējumus**, jeb, citiem vārdiem sakot, ir vajadzīga heiristiskā novērtējuma funkcija, kas dod skaitliskus novērtējumus, kas savukārt atspoguļo virsotņu kvalitāti. Turklāt heiristiski informētas pārmeklēšanas algoritmiem ir divi papildu soļi – pēc apskatāmās virsotnes izvēršanas ir nepieciešams novērtēt tās pēctečus/priekštečus (atkarībā no pārmeklēšanas virziena), pielietojot heiristiskā novērtējuma funkciju, un tad izvēlēties labāko virsotni no tiem un/vai sakārtot virsotnes atbilstoši heiristiskajiem vērtējumiem.



2. att. Heiristiski informētas pārmeklēšanas algoritmu ieejas, izejas un darbības soļi

Svarīgs ir jautājums par to, kā var izmērīt **virsotņu kvalitāti** stāvokļu telpas grafā. Pieņemsim, ir dots 3.attēlā atspoguļotais stāvokļu telpas grafs. Pārmeklēšanas sākuma virsotne ir A, pārmeklēšanas mērķa virsotne ir J. Algoritms apskata virsotni C un ģenerē tās trīs pēctečus: D, E un F. Analizējot stāvokļu telpas grafu, ir redzams, ka:

- no virsotnes E uz pārmeklēšanas mērķa virsotni ved viena loka garš ceļš (E->J);
- no virsotnes D nav ceļa uz pārmeklēšanas mērķa virsotni;
- no virsotnes F uz pārmeklēšanas mērķa virsotni ved garāki ceļi (F->E->J, F->E->H->J, F->H->J), nekā no virsotnes E.



3. att. Ģenerēto virsotņu kvalitāte, analizējot iespējas sasniegt pārmeklēšanas mērķa virsotni no šīm virsotnēm

Stāvokļu telpas grafa virsotņu kvalitāte tiek mērīta, ņemot vērā divus faktorus:

- vai n-tā virsotne vispār ļauj sasniegt pārmeklēšanas mērķa virsotni, t.i. vai n-tā virsotne atrodas uz tādiem stāvokļu telpas grafa ceļiem, kas ved uz pārmeklēšanas mērķa virsotni, un
- vai n-tā virsotne ļauj sasniegt pārmeklēšanas mērķa virsotni pa vislabāko (bieži vien īsāko) ceļu. Šī ideja abstraktā veidā tiek atspoguļota šādi (Luger, 2009):

$$f(n)=g(n)+h(n)$$

kur

- *n* – apskatāmā virsotne;
- *g(n)* – attālums no pārmeklēšanas sākuma virsotnes līdz virsotnei n; šī vērtība virza pārmeklēšanu pa īsāko ceļu;
- *h(n)* – heiristisks mērs, kas raksturo virsotnes kvalitāti jeb attāluma novērtējumu no virsotnes n līdz pārmeklēšanas mērķa virsotnei; šī vērtība vada pārmeklēšanu caur virsotnēm, kas visticamāk ved uz pārmeklēšanas mērķa virsotni.

Taču praksē augšā minētie heiristiskā novērtējuma funkcijas elementi tiek atspoguļoti netieši, fokusējoties uz tiem faktoriem, kas raksturo problēmrisināšanas stāvokļus un to nozīmību pārmeklēšanas mērķa virsotnes sasniegšanā. Rezultātā heiristiskā novērtējuma funkcija tiek izteikta kā funkcija-polinoms no *i* faktoriem (Luger, 2009):

$$f(n)=\sum a_i \cdot x_i$$

kur

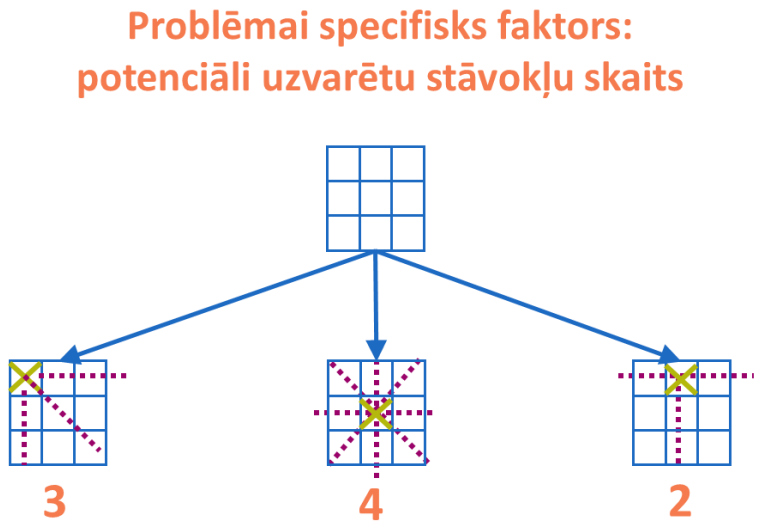
- *n* – virsotne;
- a_i – i-tā faktora svara koeficients;
- x_i – i-tā faktora, kas raksturo problēmrisināšanas stāvokļu kvalitāti, heiristisks vērtējums.

Atbilstoši 1.tabulā dotajai informācijai, funkcijā ietilpstošus faktorus un to svarus nosaka sistēmas izstrādātājs sadarbībā ar problēmsfēras ekspertiem un precizē, izmantojot mašīnmācīšanās un optimizācijas tehnikas.

Ir jāņem vērā, ka heiristiskā novērtējuma funkcija ir specifiska problēmai, jo katrai problēmai ir raksturīgas specifiskas heiristiskas zināšanas. Tā var būt ļoti vienkārša, ietverot tikai vienu vai dažus faktorus, piemēram, spēlei "Krustiņi-nullītes" mēs varam izmantot funkciju, kas nosaka potenciāli uzvarētu stāvokļu skaitu datoram (Luger, 2009). Taču funkcija var arī sastāvēt no simtiem un tūkstošiem faktoru, piemēram, šahā tiek ņemti vērā tādi faktori kā dažādu figūru kopējais skaits, bandinieku struktūra, pieejamais gājienu skaits, karaļa aktivitāte (Ma, 2012). Tādējādi izstrādājot heiristiskā novērtējuma funkciju, ir jāņem vērā:

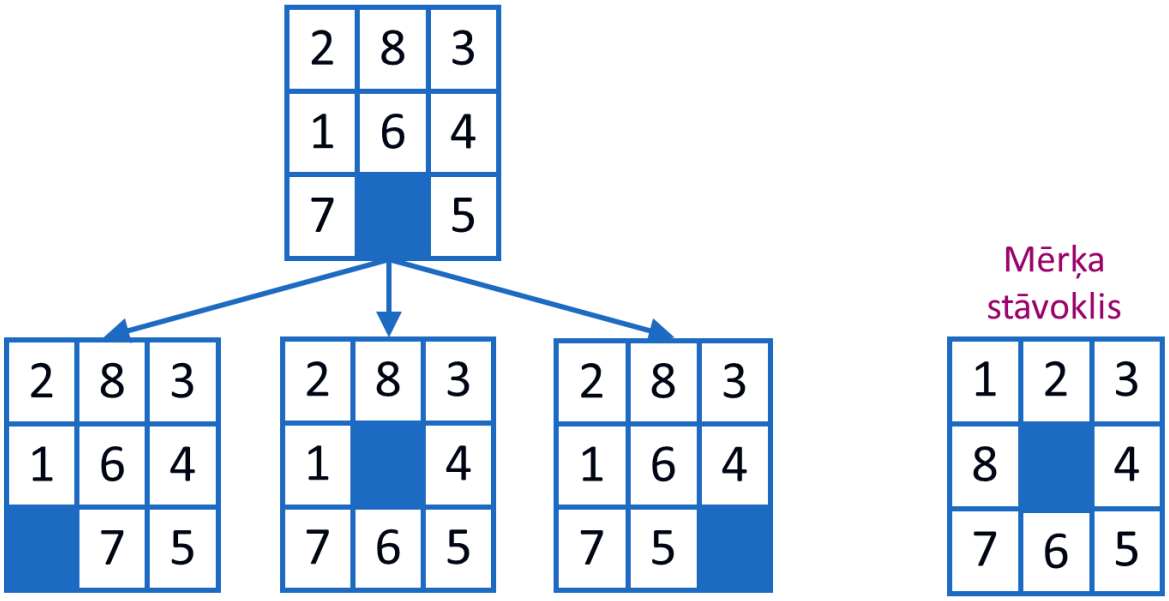
- Kādi parametri raksturo problēmrisināšanas stāvokļus stāvokļu telpas grafā?
- Kuri no identificētajiem parametriem varētu liecināt par pārmeklēšanas virzīšanos uz pārmeklēšanas mērķa virsotni?
- Kuri no identificētajiem parametriem ļautu spriest par to, vai apskatāmā virsotne atrodas uz īsākā ceļa līdz pārmeklēšanas mērķa virsotnei?

Apskatīsim spēli "Krustiņi-nullītes". Heiristiskā novērtējuma funkcija šai spēlei var tikt balstīta uz tādu faktoru kā potenciāli uzvarētu stāvokļu skaits datoram. 4.attēls atspoguļo virsotņu heiristiskus vērtējumus, izmantojot iepriekš minēto faktoru tām virsotnēm, ko dators novērtē laikā, kad tam ir jāveic gājiens: ieliekot savu zīmi kādā no stūriem, datoram paveras trīs iespējas uzvarēt, centrā – četras, sānu malā – divas.



4. att. Virsotņu heiristiskie vērtējumi spēlē "Krustiņi-nullītes"

Apskatīsim vēl vienu piemēru – spēli ar 8 kauliņiem. Spēles sākumā kauliņi ir izvietoti brīvā secībā un viens lauciņš ir tukšs. Spēlē ir jāsasniedz mērķa stāvoklis, kurā kauliņi ir izvietoti noteiktā secībā un tukšais lauciņš atrodas pa vidu. Šajā spēlē analizēsim tikai sākumstāvokļa pēctečus (5.att.).

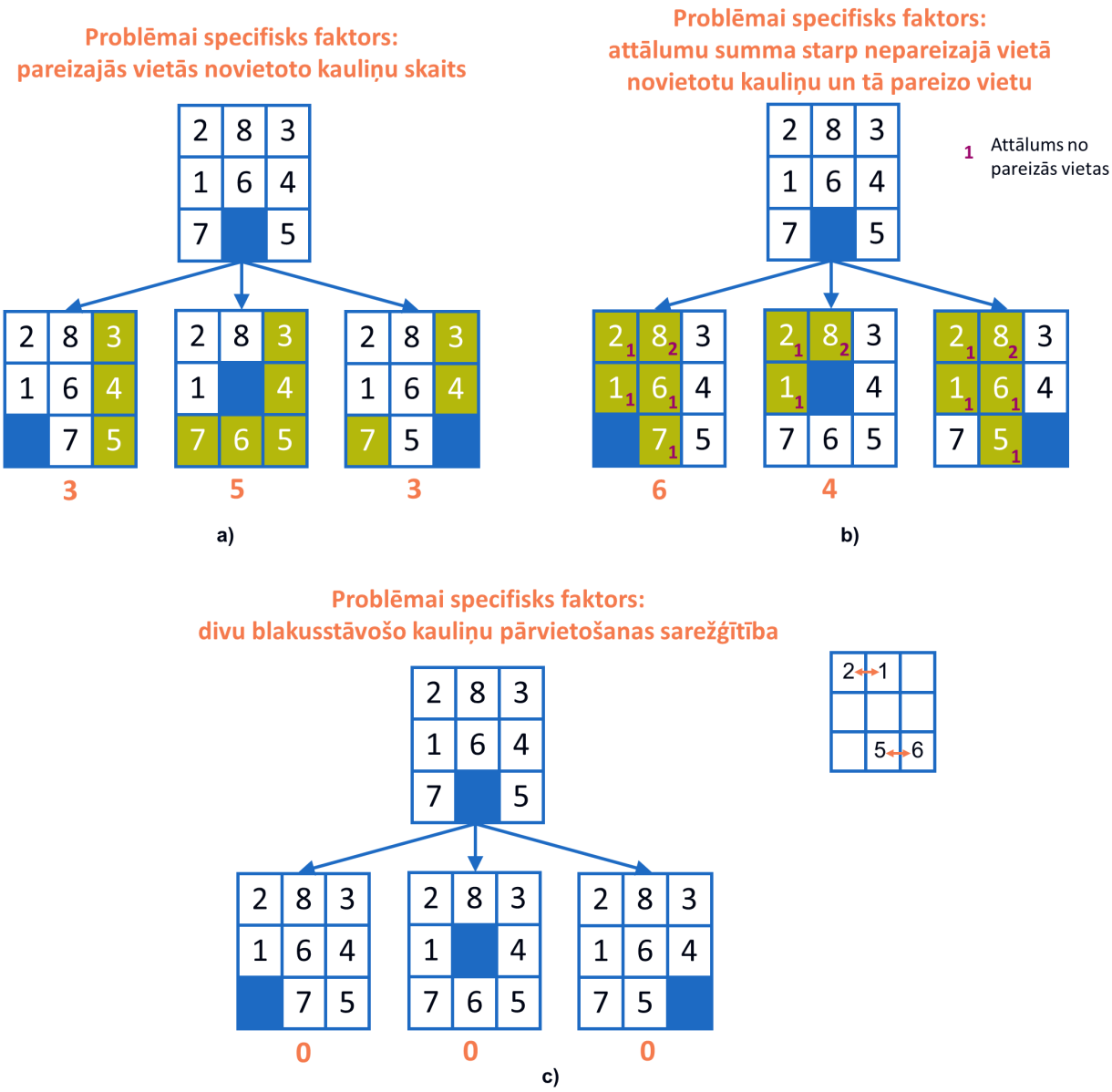


5. att. Stāvokļu telpas grafa fragments spēlei ar 8 kauliņiem

Viens no faktoriem, ko var ņemt vērā šajā spēlē, veidojot heiristiskā novērtējuma funkciju, ir pareizajās vietās novietoto kauliņu skaits (6.att. (a)). Citi faktori ir attālumu summa starp nepareizajā vietā novietotu kauliņu un tā pareizo vietu (6.att. (b)) vai divu blakusstāvošu kauliņu pārvietošanas sarežģītība (6.att. (c)), kur par sarežģītu tiek uzskatīts stāvoklis, kurā blakus atrodas kauliņi, kuri ir jāapmaina vietām, un to nevar izdarīt ar vienu gājienu.

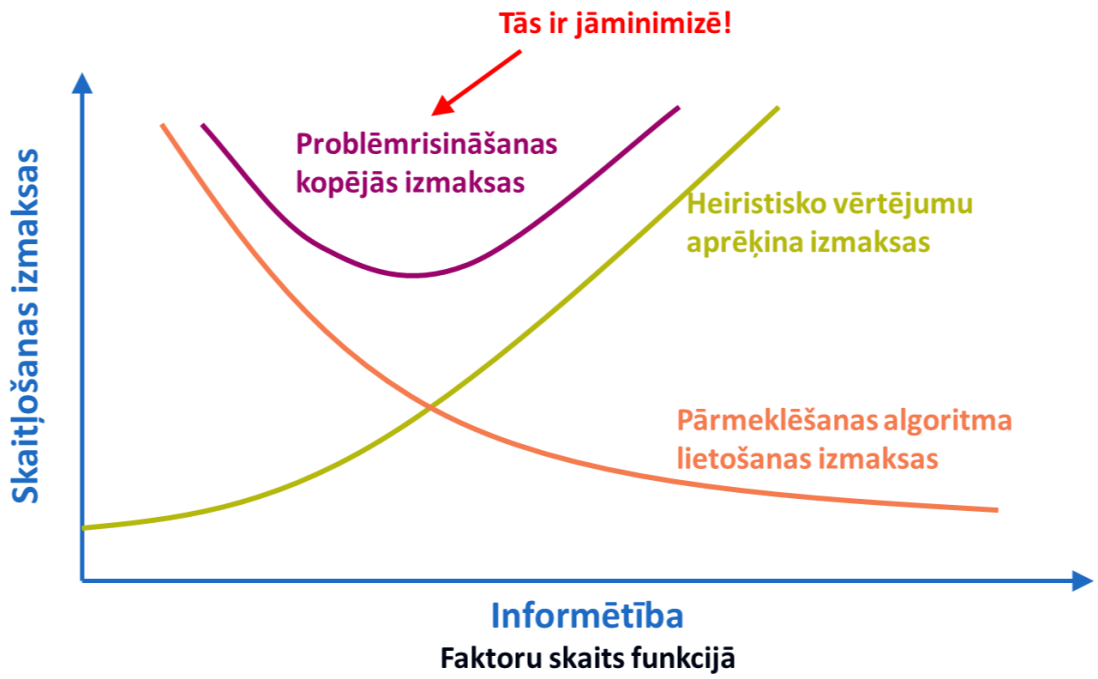
Tādējādi spēlei ar 8 kauliņiem tika identificēti trīs faktori, pamatojoties uz kuriem var definēt dažādas heiristiskā novērtējuma funkcijas:

- viena faktora funkcijas:
  - f(n) = kauliņu skaits pareizajās vietās
  - f(n) = attālumu summa
  - f(n) = blakusstāvošu kauliņu pārvietošanas sarežģītība
- divu vai trīs faktoru funkcijas, piemēram:
  - f(n) = kauliņu skaits pareizajās vietās + blakusstāvošu kauliņu pārvietošanas sarežģītība
  - f(n) = kauliņu skaits pareizajās vietās + attālumu summa + blakusstāvošu kauliņu pārvietošanas sarežģītība
- vairāku faktoru funkciju, piešķirot faktoriem svarus:
  - f(n) = svarsA*kauliņu skaits pareizajās vietās + svarsB* attālumu summa + svarsC*blakusstāvošu kauliņu pārvietošanas sarežģītība



6. att. Virsotņu heiristiskie vērtējumi spēlē ar 8 kauliņiem

Izstrādājot heiristiskā novērtējuma funkciju, ir jāatrod kompromiss starp to, cik daudz faktoru veido funkciju, un skaitļošanas resursiem, kuri tiek izmantoti, lai iegūtu funkcijas vērtības. No vienas puses, ietverot pēc iespējas vairāk faktoru funkcijā, var tikt ievērojami uzlabota pārmeklēšanas efektivitāte – ceļš, kas ved uz pārmeklēšanas mērķa virsotni, tiks atrasts ļoti ātri, apmeklējot mazāku virsotņu skaitu. No otras puses, daudz faktoru funkcijā prasīs ievērojamus skaitļošanas resursus funkcijas vērtību izskaitļošanai. To atspoguļo 7.attēls. Attēlā var redzēt, ka, palielinoties heiristikas informētībai (faktoru skaitam funkcijā), algoritmam ir jāpārmeklē mazāk virsotņu, bet tajā pašā laikā palielinās skaitļošanas izmaksas, jo sistēmai ir jārēķina heiristikā novērtējuma funkcijas vērtības. Problēmas risināšanas kopējās izmaksas ir heiristiskā novērtējuma funkcijas vērtību aprēķināšanas un stāvokļu telpas grafa pārmeklēšanas izmaksas. Šīs izmaksas ir jāminimizē. (teksta fragments balstās uz informāciju, kas ir dota (Luger, 2009))



7. att. Stāvokļu telpas grafa pārmeklēšanas izmaksas (aizgūts no (Nilsson, 1980) kā ir dots (Luger, 2009))

Bieži vien reālās pasaules problēmām stāvokļu telpas grafs ir milzīgi liels, piemēram, dambretē tas ietver ap 1040 virsotņu, šahā - 10120 virsotņu un pat spēlē "Krustiņi-nullītes", ņemot vērā, ka tā ir pietiekami vienkārša spēle, stāvokļus skaits ir ap ir 362 880 virsotņu, ja katrā līmenī tiek atspoguļotas visas (arī identiskas) virsotnes (9*8*7*6*5*4*3*2*1=9!) (Luger, 2009). Rezultātā var nepietikt skaitļošanas resursu šāda grafa glabāšanai un apstrādei. Šajā gadījumā ir jālieto heiristiskas zināšanas intelektisku datorsistēmu izstrādē.

Ir jāatzīmē, ka atkarībā no problēmas tipa pārmeklēšanas algoritmam tā darbības laikā var būt:

- Tikai daļēji pieejams stāvokļu telpas grafs ierobežotu skaitļošanas resursu dēļ. Tipiski šajā gadījumā stāvokļu telpas grafs tiek ģenerēts līdz noteiktam dziļumam intelektiskas datorsistēmas darbības laikā. Taču var veidoties tā saucamais horizonta efekts, kad dators neredz, kādas virsotnes ir aiz uzdotā dziļuma, un līdz ar to parasti izmanto heiristiskā novērtējuma funkciju, lai dators spētu izvēlēties tādas virsotnes, kas visticamāk ved uz pārmeklēšanas mērķa virsotni. Piemērs šim gadījumam ir šahs.
- Pilnībā pieejams stāvokļu telpas grafs. Tipiski šajā gadījumā ir pietiekošs skaitļošanas resursu apjoms, piemēram, plānošanas problēmās pirms aģents (piemēram, robots) sāk darboties, tiek sastādīts darbības plāns, ko attēlo kā stāvokļu telpas grafu. Šajā gadījumā heiristiskā novērtējuma funkcijas izmantošana ne vienmēr ir vajadzīga.

Vispārīgā gadījumā **stāvokļu telpas grafa sarežģītība** ir atkarīga no diviem faktoriem (Luger, 2009):

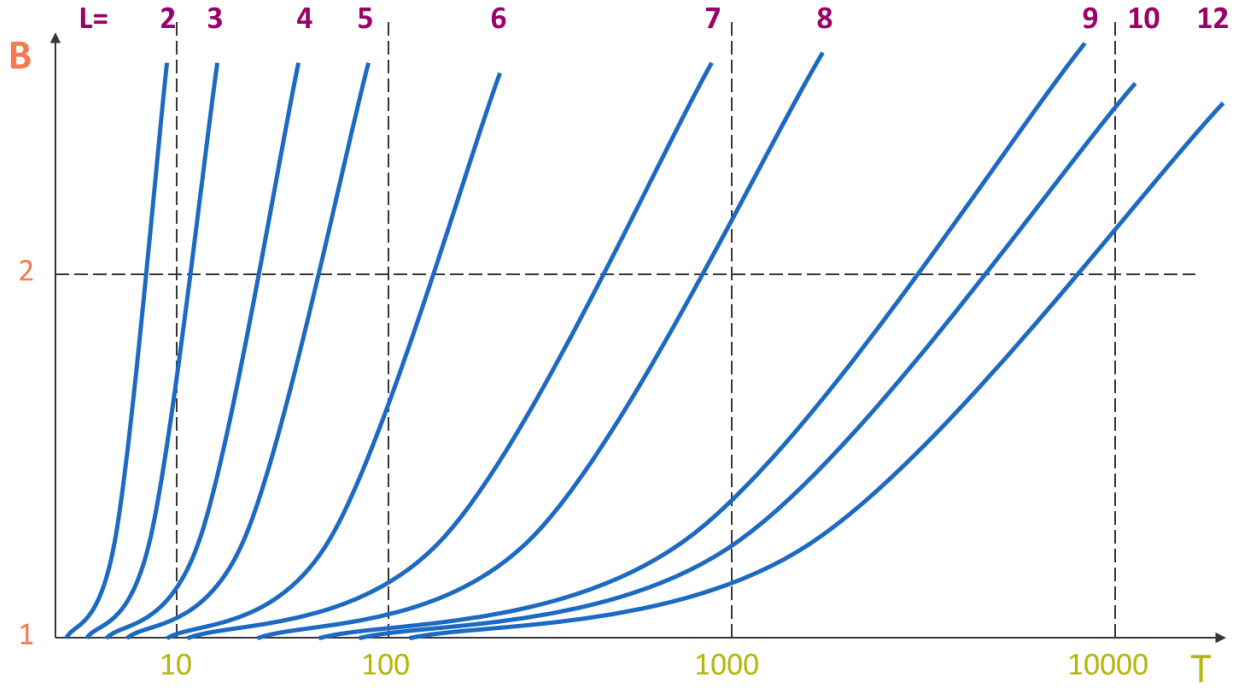
B – vidējā zarošanās koeficienta, kas ir visu virsotņu tiešo pēcteču kopējā skaita dalījums ar kopējo virsotņu skaitu stāvokļu telpas grafā;

L – stāvokļu telpas grafa dziļums (parasti garākais ceļš).

Tādējādi stāvokļu telpas grafa sarežģītību saista ar **kopējo pārmeklējamo virsotņu skaitu (T)**:

$$T = B + B^2 + B^3 + \ldots + B^L = \frac{B(B^L - 1)}{B - 1}$$

8.attēls atspoguļo stāvokļu telpas grafa vidējā zarošanās koeficienta B, dziļuma L un kopējā pārmeklējamo virsotņu skaita T savstarpējo atkarību. Tajā var redzēt, ka, ja vidējais zarošanās koeficients ir 2 (tas nozīmē, ka lielākajai daļai virsotņu ir tikai 2 pēcteči), tad, lai apskatītu visus ceļus līdz 6.dziļumam, būs jāapskata vairāk par 100 virsotnēm. Taču, palielinot dziļumu divās reizēs (līdz 12), pārmeklējamo virsotņu skaits palielinās gandrīz 10 reizēs (8190 stāvokļi). Līdz ar to var redzēt, ka pārmeklējamo virsotņu skaits aug eksponenciāli.



8. att. Pārmeklējamo virsotņu skaita (T) atkarība no vidējā zarošanās koeficienta (B) un stāvokļu telpas grafa dziļuma (L) (aizgūts no (Nilsson, 1980) kā ir dots (Luger, 2009))

Būtībā ir divas metodes stāvokļu telpas grafa sarežģītības samazināšanai:

- informētāku heiristiskā novērtējuma funkciju izmantošana. Šajā gadījumā heiristiskā novērtējuma funkcijā ir jāietver pēc iespējas vairāk faktoru, tomēr ir jāatceras par izmaksu kompromisu, kas tika atspoguļots 7.attēlā;
- stāvokļu telpas grafa transformācija. Diemžēl šai metodei nav precīzu vadlīniju, kā veikt transformāciju. Viena no piedāvātajām pieejām ir ņemt vērā simetriskuma pazīmi, kā to var darīt spēlē “Krustiņi-nullītes”, sākumā apskatot tikai trīs iespējamus gājienus: stūrī, centrā un sānu malā. Tas ļauj samazināt pārmeklējamo virsotņu skaitu no 362880 (ja tiek atspoguļotas visas virsotnes) līdz 12*7!=60480 virsotnēm (Luger, 2009).

Tādējādi heiristisku zināšanu izmantošana stāvokļu telpas grafa pārmeklēšanā var ievērojami uzlabot pārmeklēšanas efektivitāti. Taču šim nolūkam ir vajadzīga heiristiskā novērtējuma funkcija, kuru izstrādājot ir jāievēro kompromiss starp pārmeklēšanas algoritma lietošanas izmaksām un heiristiskā novērtējuma funkcijas vērtību aprēķināšanas izmaksām, kas pēc būtības nav triviāls

uzdevums. Tieši tāpēc, izstrādājot heiristiskā novērtējuma funkciju, intelektiskas datorsistēmas izstrādātājam ir jāsadarbojas ar problēmsfēras ekspertiem, lai no viņiem iegūtu heiristiskas zināšanas, un tad, pēc funkcijas izveides, tā ir jāprecizē, izmantojot mašīnmācīšanās un optimizācijas tehnikas.

3.2. tēma: Heiristiski informētas pārmeklēšanas algoritmi

Tēmas mērķis ir izveidot studentu zināšanu sistēmu par trim heiristiski informētas pārmeklēšanas algoritmiem un attīstīt to realizācijas prasmes.

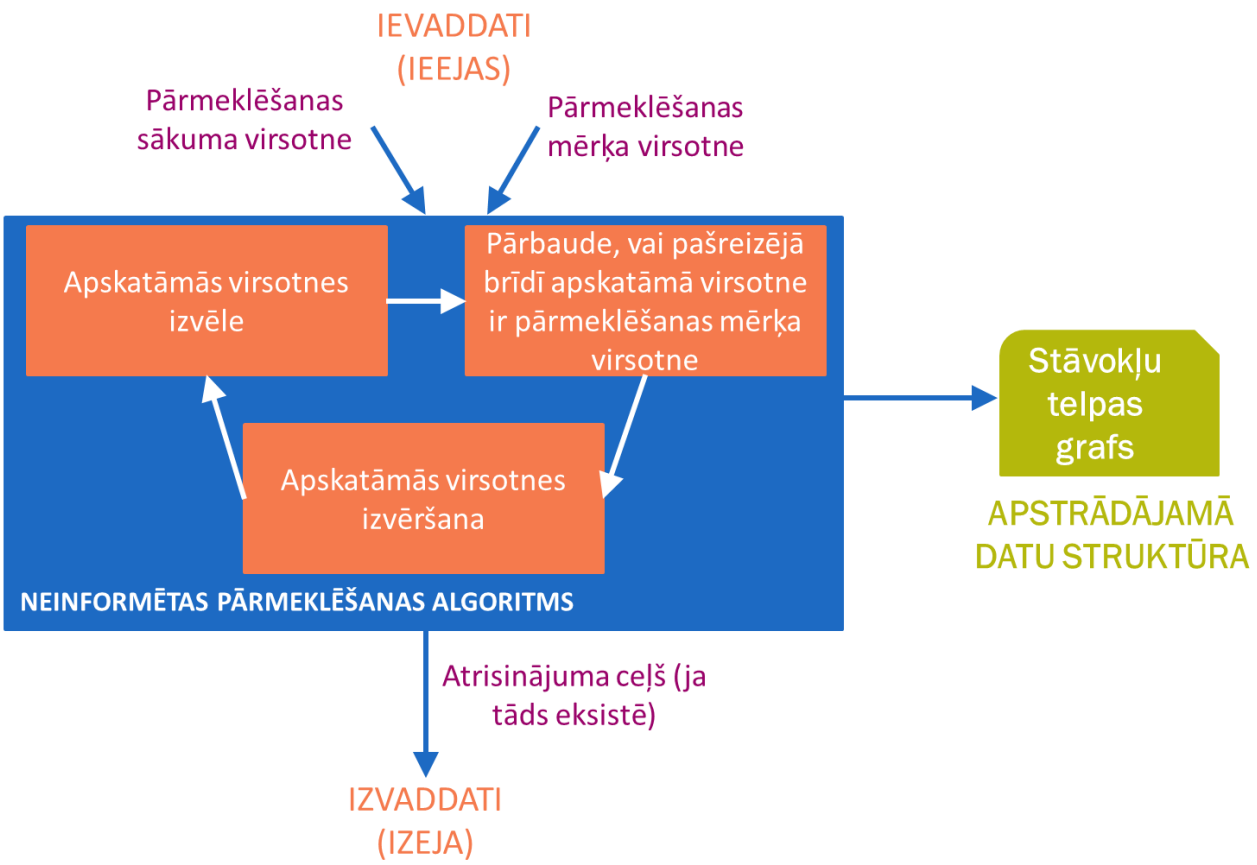
Sasniedzamie studiju rezultāti - Pēc šīs tēmas apgūšanas studenti:

- zinās kalnā kāpšanas, vislabākā stāvokļa meklēšanas un starveida pārmeklēšanas algoritmu darbības principus, kā arī šo algoritmu priekšrocības un trūkumus;
- pratīs demonstrēt minēto algoritmu darbības realizāciju ar sarakstu OPEN un CLOSED palīdzību.

Iepriekšējā tēmā mēs apspriedām, kā pārmeklēšanas efektivitāti stāvokļu telpas grafā var ievērojami uzlabot, izmantojot heiristiskas jeb pieredzē-balstītas zināšanas. Turklāt pārmeklēšanas efektivitāte tiek saistīta ar to, ka, izmantojot heiristiskas zināšanas, algoritmi apskata mazāk virsotņu stāvokļu telpas grafā un rezultātā samazina arī kopējo pārmeklēšanas laiku. Šajā tēmā apskatīsim konkrētus heiristiski informētas pārmeklēšanas algoritmus – to idejas un realizācijas aspektus. Taču pirms runāt par algoritmiem, es vēlētos jums atgādināt neinformētas un heiristiski informētas pārmeklēšanas algoritmu darbības principus, lai Jums būtu skaidrs priekšstats par atšķirībām starp šīm divām pārmeklēšanas algoritmu kategorijām.

Tādējādi neinformētas pārmeklēšanas algoritmi saņem kā ieejas datus pārmeklēšanas sākuma virsotni un pārmeklēšanas mērķa virsotni, kā resursu – stāvokļu telpas grafu, kurā tie veic pārmeklēšanu, un rezultātā tie izdod atrisinājuma ceļu (1.att.). Algoritmu pamatā ir trīs soļi:

1. noteikt apskatāmo virsotni, kas visbiežāk ir virsotne OPEN saraksta pirmajā pozīcijā;
2. pārbaudīt, vai apskatāmā virsotne ir pārmeklēšanas mērķa virsotne;
3. izvērst apskatāmo virsotni.



1. att. Neinformētas pārmeklēšanas algoritmu ieejas, izejas un darbības soļi

Savukārt, **heiristiski informētas pārmeklēšanas algoritmi** saņem kā ieejas datus arī pārmeklēšanas sākuma virsotni un pārmeklēšanas mērķa virsotni, bet papildus tiem ir vajadzīgi heiristisko vērtējumu aprēķina principi (2.att.). Šie algoritmi arī izmanto stāvokļu telpas grafu kā resursu un izdod atrisinājuma ceļu kā rezultātu. Taču tiem pamatā ir daļēji atšķirīgi soļi:

1. noteikt apskatāmo virsotni;
2. pārbaudīt, vai apskatāmā virsotne ir pārmeklēšanas mērķa virsotne;
3. izvērst apskatāmo virsotni;
4. novērtēt apskatāmās virsotnes pēctečus/priekštečus (atkarībā no pārmeklēšanas virziena);
5. Izvēlēties labākās virsotnes un/vai sakārtot virsotnes atbilstoši heiristiskajiem vērtējumiem.

Virsotnes kopā ar heiristiskajiem vērtējumiem

Iterācija	Open	Closed

Virsotņu sakārtošana atbilstoši heiristiskajiem vērtējumiem

3. att. Pārmeklēšanas algoritmu darbības atspoguļošanas veids

(IZEJA)

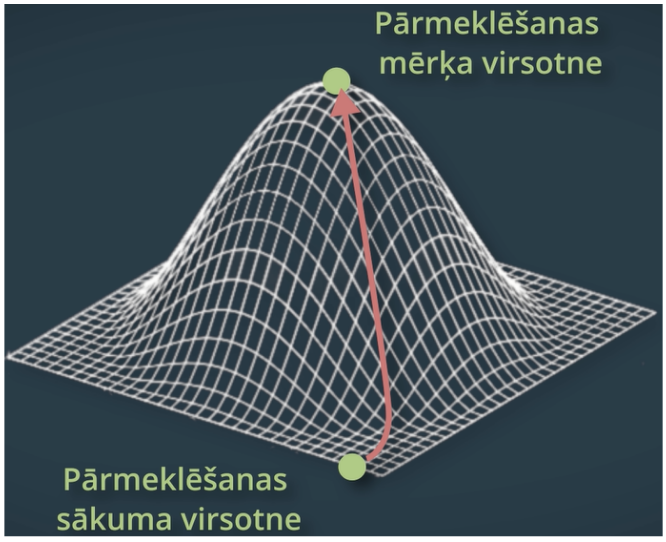
2. att. Heiristiski informētas pārmeklēšanas algoritmu ieejas, izejas un darbības soļi

Heiristiski informētas pārmeklēšanas algoritmus realizē, izmantojot Jums jau labi zināmus sarakstus OPEN un CLOSED, kurus mēs šī studiju kursa ietvaros turpināsim atspoguļot tabulas veidā (3.att.).

Ar šiem sarakstiem ir saistīti šādi svarīgi aspekti:

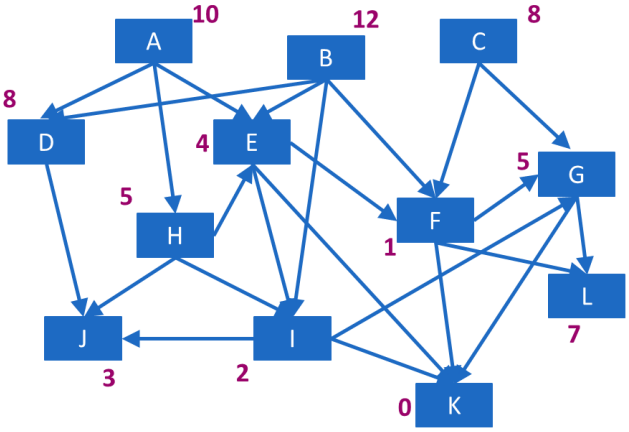
- abos sarakstos virsotnes ievieto kopā ar to heiristiskajiem vērtējumiem;
- sarakstā OPEN virsotnes parasti sakārto atbilstoši to heiristiskajiem vērtējumiem no virsotnes ar labāko vērtējumu uz virsotni ar sliktāko vērtējumu.

Kalnā kāpšanas algoritma ideja ir vienmēr virzīties uz labāko virsotni stāvokļu telpas grafā. Ja mēs iedomāsimies, ka stāvokļu telpas grafs, kur katrai virsotnei ir definēts heiristisks vērtējums, ir līdzīgs kalnam, tad pārmeklēšanas mērķa virsotne var tikt uzskatīta par kalna virsotni, bet pārmeklēšanas sākuma virsotne par kalna pakāji (4.att.). Šajā gadījumā kalnā kāpšanas algoritms kāps augšā, uz labāko heiristisko vērtējumu, kas ir definēts pārmeklēšanas mērķa virsotnei.



4.att. Kalnā kāpšanas algoritma darbības atspoguļojums

Šis algoritms vienmēr sarakstā OPEN glabā vienu labāko virsotni, kuru arī izvērš kārtējā iterācijā. Apskatāmās virsotnes pēctečus/priekštečus (atkarībā no pārmeklēšanas virziena) tas novērtē, izvēlas no tiem labāko, ievietojot to sarakstā OPEN, un pārējās izvērstās virsotnes ievieto sarakstā CLOSED un nekad tās vairs neapskata. Tādējādi šis algoritms nerealizē atkāpšanos. Kalnā kāpšana beidz darbību, vai nu tad, kad ir sasniegta pārmeklēšanas mērķa virsotne, vai kad ir sasniegta virsotne, kuras heiristiskais vērtējums ir labāks nekā šīs virsotnes pēcteču/priekšteču (atkarībā no pārmeklēšanas virziena) heiristiskie vērtējumi.



No datiem virzītā kalnā kāpšana

Pārmeklēšanas sākuma virsotne – A

Pārmeklēšanas mērķa virsotne – K

Heiristiskie vērtējumi ir norādīti blakus virsotnēm

Labākais heiristiskais vērtējums ir mazākais

Iterācija	Open	Closed
0.	A(10)	-
1.	E(4)	A(10) D(8) H(5)
2.	K(0)	A(10) D(8) H(5) E(4) F(1) I(2)

Pārmeklēšanas mērķa virsotne ir sasniegta

5. att. Kalnā kāpšanas realizācijas piemērs

Apskatīsim kalnā kāpšanas realizācijas piemēru, izpildot no datiem virzītu pārmeklēšanu 5.attēlā redzamajā grafā, kur pārmeklēšanas sākuma virsotne ir virsotne A, pārmeklēšanas mērķa virsotne ir virsotne K, virsotņu heiristiskie vērtējumi ir norādīti blakus virsotnēm, un ir pieņemts, ka labākais heiristiskais vērtējums ir mazākais.

0.iterācijā pārmeklēšanas sākuma virsotne kopā ar tās heiristisko vērtējumu tiek ievietota sarakstā OPEN, jo tā kļūst par pirmo apskatāmo virsotni. Saraksts CLOSED ir tukšs, jo neviena virsotne vēl netika izvērsta. Algoritms paņem virsotni, kas atrodas sarakstā OPEN – virsotni A, un pārbauda, vai tā ir pārmeklēšanas mērķa virsotne. Tā kā tā nav pārmeklēšanas mērķa virsotne, tiek uzsākta nākamā iterācija.

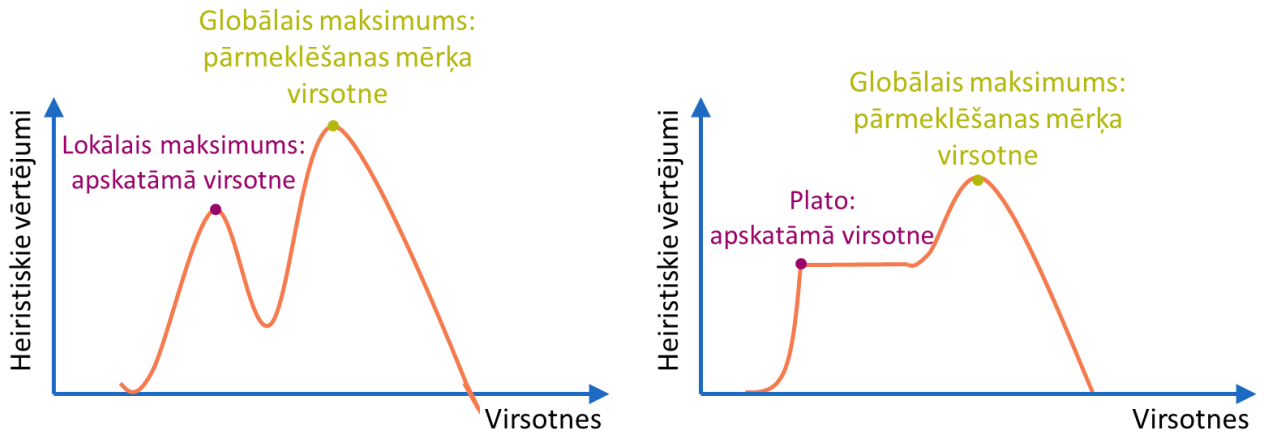
1.iterācijā algoritms izvērš apskatāmo virsotni A, ģenerējot tās trīs pēctečus – virsotnes D, H, un E. Tā kā neviena no šīm virsotnēm nav ne sarakstā OPEN, ne sarakstā CLOSED, tad algoritms iegūst visu šo virsotņu heiristiskos vērtējumus – 8, 5 un 4 attiecīgi un, pamatojoties uz tiem, izvēlas labāko virsotni, kuras heiristiskais vērtējums ir vismazākais no iegūtajiem vērtējumiem. Tā ir virsotne E ar vērtējumu 4, kas tiek ievietota sarakstā OPEN. Pārējās divas virsotnes kopā ar jau apskatīto un izvērsto virsotni A tiek pārvietotas sarakstā CLOSED. Algoritms paņem virsotni, kas atrodas sarakstā OPEN – virsotni E, un pārbauda, vai tā ir pārmeklēšanas mērķa virsotne. Tā kā tā nav pārmeklēšanas mērķa virsotne, tiek uzsākta nākamā iterācija.

2.iterācijā algoritms izvērš apskatāmo virsotni E, ģenerējot tās trīs pēctečus – virsotnes I, K, un F. Tā kā neviena no šīm virsotnēm nav ne sarakstā OPEN, ne sarakstā CLOSED, tad algoritms iegūst visu šo virsotņu heiristiskos vērtējumus – 2, 0 un 1 attiecīgi un, pamatojoties uz tiem, izvēlas labāko virsotni, kuras heiristiskais vērtējums ir vismazākais no iegūtajiem vērtējumiem. Tā ir virsotne K ar

vērtējumu 0, kas tiek ievietota sarakstā OPEN. Pārējās divas virsotnes kopā ar jau apskatīto un izvērsto virsotni E tiek pārvietotas sarakstā CLOSED. Algoritms paņem virsotni, kas atrodas sarakstā OPEN – virsotni K, un pārbauda, vai tā ir pārmeklēšanas mērķa virsotne. Tā kā tā ir pārmeklēšanas mērķa virsotne, algoritms beidz darbību, paziņojot, ka pārmeklēšanas mērķa virsotne ir sasniegta.

No vienas puses, kalnā kāpšanas algoritms spēj ļoti ātri atrast pārmeklēšanas mērķa virsotni, ja heiristiskā novērtējuma funkcija ir pietiekami labi definēta. No otras puses, ja funkcija nav visai labi definēta, tad kalnā kāpšana var novest pie vairākiem trūkumiem, no kuriem izplatītākie ir šādi (6.att.):

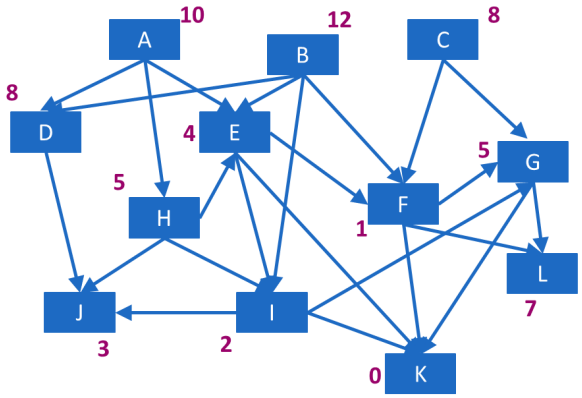
- **lokālā maksimuma problēma**: algoritms nesasniedz pārmeklēšanas mērķa virsotni, jo pārmeklēšanas laikā atrod virsotni, kuras heiristiskais vērtējums ir labāks par tās pēcteču/priekšteču heiristiskajiem vērtējumiem;
- **plato problēma**: algoritms nespēj turpināt pārmeklēšanu, ja apskatāmās virsotnes labākā pēcteča/priekšteča heiristiskais vērtējums ir vienāds ar apskatāmās virsotnes heiristisko vērtējumu.



6. att. Kalnā kāpšanas algoritma trūkumi

Vislabākā stāvokļa meklēšanas algoritms katrā iterācijā apskata vienu labāko virsotni, taču neaizmirst arī pārējās, kuru heiristiskie vērtējumi nav tik labi. Tas ļauj algoritmam, pirmkārt, pārslēgties no viena stāvokļu telpas grafa zara uz citu, un, otrkārt, nezaudēt atrisinājumus pat tad, ja heiristiskā novērtējuma funkcija nav pietiekami labi definēta.

Tādējādi šis algoritms sarakstā OPEN saglabā visas atrastas virsotnes un katrā iterācijā izvērš virsotni, kas atrodas saraksta OPEN pirmajā pozīcijā. Ģenerējot un izvērtējos tās pēcteču/priekšteču (atkarībā no pārmeklēšanas virziena), šis algoritms ievieto tos sarakstā OPEN, vienlaicīgi veicot saraksta šķirošanu atbilstoši virsotņu heiristiskajiem vērtējumiem – no virsotnes ar labāko vērtējumu uz virsotni ar sliktāko vērtējumu. Šajā algoritmā OPEN saraksts tiek realizēts kā prioritātes rinda, kas darbojas pēc principa – kam ir labāks vērtējums, tas pirmais tiek apskatīts.



No datiem virzītā vislabākā stāvokļa meklēšana

Pārmeklēšanas sākuma virsotne – A

Pārmeklēšanas mērķa virsotne – J

Heiristiskie vērtējumi ir norādīti blakus virsotnēm

Labākais heiristiskais vērtējums ir mazākais

Iterācija	Open	Closed
0.	A(10)	-
1.	E(4) H(5) D(8)	A(10)
2.	K(0) F(1) I(2) H(5) D(8)	A(10) E(4)
3.	F(1) I(2) H(5) D(8)	A(10) E(4) K(0)
4.	I(2) H(5) G(5) L(7) D(8)	A(10) E(4) K(0) F(1)
5.	J(3) H(5) G(5) L(7) D(8)	A(10) E(4) K(0) F(1) I(2)

Pārmeklēšanas mērķa virsotne ir sasniegta

7. att. Vislabākā stāvokļa meklēšanas realizācijas piemērs

Apskatīsim vislabākā stāvokļa meklēšanas algoritma realizācijas piemēru, izpildot no datiem virzītu pārmeklēšanu 7.attēlā redzamajā grafā, kur pārmeklēšanas sākuma virsotne ir virsotne A, pārmeklēšanas mērķa virsotne ir virsotne J, virsotņu heiristiskie vērtējumi ir norādīti blakus virsotnēm, un ir pieņemts, ka labākais heiristiskais vērtējums ir mazākais.

0.iterācijā pārmeklēšanas sākuma virsotne kopā ar tās heiristisko vērtējumu tiek ievietota sarakstā OPEN, jo tā kļūst par pirmo apskatāmo virsotni. Saraksts CLOSED ir tukšs, jo neviena virsotne vēl netika izvērsta. Algoritms paņem virsotni, kas atrodas saraksta OPEN pirmajā pozīcijā – virsotni A, un pārbauda, vai tā ir pārmeklēšanas mērķa virsotne. Tā kā tā nav pārmeklēšanas mērķa virsotne, tiek uzsākta nākamā iterācija.

1.iterācijā algoritms izvērš apskatāmo virsotni A, ģenerējot tās trīs pēctečus – virsotnes D, H, un E. Tā kā neviena no šīm virsotnēm nav ne sarakstā OPEN, ne sarakstā CLOSED, tad algoritms iegūst visu šo virsotņu heiristiskos vērtējumus – 8, 5 un 4 attiecīgi un ievieto ģenerētās virsotnes sarakstā OPEN, tās sakārtojot atbilstoši heiristiskajiem vērtējumiem no virsotnes ar labāko vērtējumu uz virsotni ar sliktāko vērtējumu. Savukārt virsotne A kā jau apskatīta un izvērsta virsotne tiek pārvietota sarakstā CLOSED. Algoritms paņem virsotni, kas atrodas saraksta OPEN pirmajā pozīcijā – virsotni E, un pārbauda, vai tā ir pārmeklēšanas mērķa virsotne. Tā kā tā nav pārmeklēšanas mērķa virsotne, tiek uzsākta nākamā iterācija.

2.iterācijā algoritms izvērš apskatāmo virsotni E, ģenerējot tās trīs pēctečus – virsotnes I, K, un F. Tā kā neviena no šīm virsotnēm nav ne sarakstā OPEN, ne sarakstā CLOSED, tad algoritms iegūst visu šo virsotņu heiristiskos vērtējumus – 2, 0 un 1 attiecīgi un ievieto ģenerētās virsotnes sarakstā OPEN, vienlaicīgi pārkārtojot virsotnes šajā sarakstā atbilstoši heiristiskajiem vērtējumiem no virsotnes ar labāko vērtējumu uz virsotni ar sliktāko vērtējumu. Savukārt virsotne E kā jau apskatīta un izvērsta virsotne tiek pārvietota sarakstā CLOSED. Algoritms paņem virsotni, kas atrodas saraksta OPEN pirmajā pozīcijā – virsotni K, un pārbauda, vai tā ir pārmeklēšanas mērķa virsotne. Tā kā tā nav pārmeklēšanas mērķa virsotne, tiek uzsākta nākamā iterācija.

3.iterācijā algoritms mēģina izvērst apskatāmo virsotni K, ģenerējot tās pēctečus, bet tādu nav, jo šī ir strupceļa virsotne. Līdz ar to jaunas virsotnes netiek pievienotas sarakstam OPEN, bet virsotne K

kā jau apskatīta un izvērsta virsotne tiek pārvietota sarakstā CLOSED. Algoritms paņem virsotni, kas atrodas saraksta OPEN pirmajā pozīcijā – virsotni F, un pārbauda, vai tā ir pārmeklēšanas mērķa virsotne. Tā kā tā nav pārmeklēšanas mērķa virsotne, tiek uzsākta nākamā iterācija.

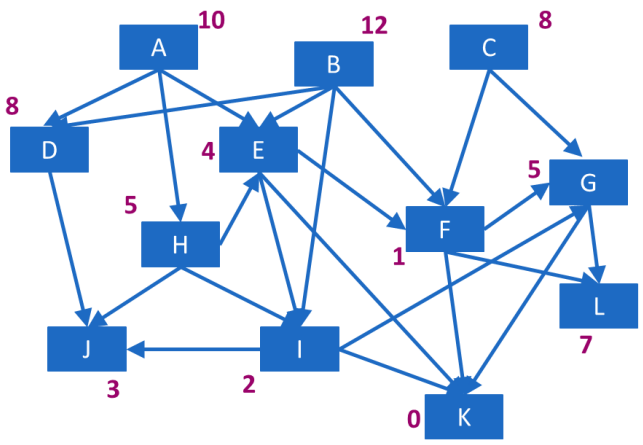
4.iterācijā algoritms izvērš apskatāmo virsotni F, ģenerējot tās trīs pēctečus – virsotnes K, L un G. Tā kā virsotne K jau ir sarakstā CLOSED, to atkārtoti neapskata. Algoritms iegūst virsotņu L un G heiristiskos vērtējumus – 7 un 5 attiecīgi un ievieto ģenerētās virsotnes sarakstā OPEN, vienlaikus pārkārtojot virsotnes šajā sarakstā atbilstoši heiristiskajiem vērtējumiem no virsotnes ar labāko vērtējumu uz virsotni ar sliktāko vērtējumu. Savukārt virsotne F kā jau apskatīta un izvērsta virsotne tiek pārvietota sarakstā CLOSED. Algoritms paņem virsotni, kas atrodas saraksta OPEN pirmajā pozīcijā – virsotni I, un pārbauda, vai tā ir pārmeklēšanas mērķa virsotne. Tā kā tā nav pārmeklēšanas mērķa virsotne, tiek uzsākta nākamā iterācija.

5.iterācijā algoritms izvērš apskatāmo virsotni I, ģenerējot tās trīs pēctečus – virsotnes J, K un G. Tā kā virsotne K jau ir sarakstā CLOSED un virsotne G – sarakstā OPEN, tās atkārtoti neapskata. Algoritms iegūst virsotnes J heiristisko vērtējumu, kas ir vienāds ar 3, un ievieto šo virsotni sarakstā OPEN, vienlaikus pārkārtojot virsotnes šajā sarakstā atbilstoši heiristiskajiem vērtējumiem no labākās virsotnes uz sliktāko virsotni. Savukārt virsotne I kā jau apskatīta un izvērsta virsotne tiek pārvietota sarakstā CLOSED. Algoritms paņem virsotni, kas atrodas saraksta OPEN pirmajā pozīcijā – virsotni J, un pārbauda, vai tā ir pārmeklēšanas mērķa virsotne. Tā kā tā ir pārmeklēšanas mērķa virsotne, algoritms beidz darbību, paziņojot, ka pārmeklēšanas mērķa virsotne ir sasniegta.

Tādējādi šis algoritms vienmēr atrod pārmeklēšanas mērķa virsotni, ja tāda eksistē stāvokļu telpas grafā, bet skaitļošanas resursu patēriņš, it īpaši ja heiristiskā novērtējuma funkcija nav visai labi definēta, var būt ievērojami lielāks nekā pārējiem heiristiski informētas pārmeklēšanas algoritmiem.

Starveida pārmeklēšana ir kompromiss starp kalnā kāpšanu, kurā tiek apskatīta tikai viena labākā virsotne, un vislabākā stāvokļa meklēšanu, kurā tiek saglabātas un pakāpeniski apskatītas visas ģenerētas virsotnes. Šis algoritms katrā iterācijā izvēlas tālākajai izvēršanai ierobežotu skaitu labāko virsotņu. Taču tas nerealizē atkāpšanos.

Tādējādi pirms algoritma izpildes tiek uzdots stara platums jeb skaitlis, kas nosaka, cik daudz labāko virsotņu tiks apskatītas katrā iterācijā. Katrā iterācijā algoritms izvērš vienlaicīgi visas virsotnes, kas atrodas sarakstā OPEN. Visu izvērsto virsotņu priekšteči/pēcteči (atkarībā no pārmeklēšanas virziena) tiek novērtēti, un no tiem izvēlas un ievieto sarakstā OPEN labākos atbilstoši stara platumam, pārējās virsotnes ievietojot sarakstā CLOSED. Rezultātā saraksts OPEN satur tik daudz virsotņu, cik ir stara platums, izņemot 0.iterāciju, kad OPEN sarakstā tiek ievietota tikai pārmeklēšanas sākuma virsotne, un gadījumu, kad ģenerēto pēcteču/priekšteču skaits ir mazāks nekā stara platums. Algoritms beidz darbību, kad pārmeklēšanas mērķa virsotne ir ievietota sarakstā OPEN vai arī kad ir sasniegtas strupceļa virsotnes, kas nav pārmeklēšanas mērķa virsotne.



No datiem virzītā starveida pārmeklēšana

Pārmeklēšanas sākuma virsotne – A

Pārmeklēšanas mērķa virsotne – K

Stara platums – 2 virsotnes

Heiristiskie vērtējumi ir norādīti blakus virsotnēm

Labākais heiristiskais vērtējums ir mazākais

Iterācija	Open	Closed
0.	A(10)	-
1.	E(4) H(5)	A(10) D(8)
2.	K(0) F(1)	A(10) D(8) E(4) H(5) I(2) J(3)

Pārmeklēšanas mērķa virsotne ir sasniegta

8. att. Starveida pārmeklēšanas realizācijas piemērs

Apskatīsim starveida pārmeklēšanas algoritma realizācijas piemēru, izpildot no datiem virzītu pārmeklēšanu 8.attēlā redzamajā grafā, kur pārmeklēšanas sākuma virsotne ir virsotne A, pārmeklēšanas mērķa virsotne ir virsotne K, stara platums ir 2 virsotnes, virsotņu heiristiskie vērtējumi ir norādīti blakus virsotnēm, un ir pieņemts, ka labākais heiristiskais vērtējums ir mazākais.

0.iterācijā pārmeklēšanas sākuma virsotne kopā ar tās heiristisko vērtējumu tiek ievietota sarakstā OPEN, jo tā kļūst par pirmo apskatāmo virsotni. Saraksts CLOSED ir tukšs, jo neviena virsotne vēl netika izvērsta. Algoritms paņem virsotni, kas atrodas sarakstā OPEN – virsotni A, un pārbauda, vai tā ir pārmeklēšanas mērķa virsotne. Tā kā tā nav pārmeklēšanas mērķa virsotne, tiek uzsākta nākamā iterācija.

1.iterācijā algoritms izvērš apskatāmo virsotni A, ģenerējot tās trīs pēcteču – virsotnes D, H, un E. Tā kā neviena no šīm virsotnēm nav ne sarakstā OPEN, ne sarakstā CLOSED, tad algoritms iegūst visu šo virsotņu heiristiskos vērtējumus – 8, 5 un 4 attiecīgi un, atbilstoši stara platumam, izvēlas no tiem divas labākās virsotnes, pamatojoties uz virsotņu heiristiskajiem vērtējumiem. Tās ir virsotnes E un H, kas tiek ievietotas sarakstā OPEN sakārtotā veidā no virsotnes ar labāko heiristisko vērtējumu uz virsotni ar sliktāko vērtējumu. Savukārt virsotne A kā jau apskatīta un izvērsta virsotne kopā ar virsotni D, kas tika ģenerēta, bet netika izvēlēta tālākajai apskatīšanai, tiek pārvietotas sarakstā CLOSED. Algoritms paņem abas virsotnes, kas atrodas sarakstā OPEN, – virsotnes E un H, un pārbauda, vai starp tām ir pārmeklēšanas mērķa virsotne. Tā kā pārmeklēšanas mērķa virsotne nav atrasta, tiek uzsākta nākamā iterācija.

2.iterācijā Algoritms vienlaikus izvērš abas virsotnes, kas atrodas sarakstā OPEN, ģenerējot to pēcteču. Virsotnei E pēcteči ir virsotnes I, K un F. Virsotnes H pēcteči ir J, I un E. Tā kā virsotne E jau atrodas sarakstā OPEN, to izslēdz no tālākās apskatīšanas un nepievieno nevienam no sarakstiem. No atlikušajām virsotnēm paliek četras unikālas virsotnes I, K, F un J. Tā kā neviena no šīm virsotnēm nav ne sarakstā OPEN, ne sarakstā CLOSED, tad algoritms iegūst visu šo virsotņu heiristiskos vērtējumus – 2, 0, 1 un 3 attiecīgi un, atbilstoši stara platumam, izvēlas no tiem divas labākās virsotnes, pamatojoties uz virsotņu heiristiskajiem vērtējumiem. Tās ir virsotnes K un F, kas tiek ievietotas sarakstā OPEN sakārtotā veidā no virsotnes ar labāko heiristisko vērtējumu uz virsotni ar sliktāko vērtējumu. Savukārt virsotnes E un H kā jau apskatītas un izvērstas virsotnes

kopā ar virsotnēm I un J, kas tika ģenerētas, bet netika izvēlētas tālākajai apskatīšanai, tiek pārvietotas sarakstā CLOSED. Algoritms paņem virsotnes, kas atrodas sarakstā OPEN – virsotnes K un F, un pārbauda, vai starp tām ir pārmeklēšanas mērķa virsotne. Tā kā pārmeklēšanas mērķa virsotne atrodas sarakstā OPEN, algoritms beidz darbību, paziņojot, ka pārmeklēšanas mērķa virsotne ir sasniegta.

Kaut gan šis algoritms var novest pie pārmeklēšanas mērķa virsotnes ātrāk nekā vislabākā stāvokļa meklēšana, tam nav nekādu vadlīniju, kā būtu jāizvēlas stara platums.

Šajā tēmā esam apskatījuši trīs heiristiski informētas pārmeklēšanas algoritmu idejas un realizācijas aspektus. Lūdzu atcerēties, ka šo algoritmu efektivitāte ir atkarīga no heiristiskā novērtējuma funkcijas kvalitātes: jo labāk definēta ir šī funkcija, jo ātrāk algoritmi varēs atrast pārmeklēšanas mērķa virsotni. Taču, ja funkcija nav visai labi definēta, tad katram no šiem algoritmiem var izpausties tā trūkumi.

3.3. tēma: Heiristiski informētas pārmeklēšanas algoritmi spēļu izstrādei

Tēmas mērķis ir izveidot studentu zināšanu sistēmu par heiristiski informētas pārmeklēšanas algoritmiem deterministisku divpersonu spēļu ar pilnu informāciju realizācijai un attīstīt studentu prasmes realizēt šos algoritmus.

Sasniedzamie studiju rezultāti - Pēc šīs tēmas apgūšanas studenti:

- zinās, kā spēles koka pieejamība ietekmē algoritma izvēli deterministiskai divpersonu spēlei ar pilnu informāciju izstrādei;
- pratīs pielietot Minimaksa un Alfa-beta algoritmus divpersonu spēļu ar pilnu informāciju realizācijai;
- zinās Pārlūkošanas uz priekšu pār *n*-gājieniem un Iteratīvās iedziļināšanas metožu būtību.

Divpersonu spēļu ar pilnu informāciju realizācija uzskatāmi demonstrē visās iepriekšējās tēmās apgūto jēdzienu praktisko lietojumu. Tātad spēļu realizācijā tiek izmantots spēles koks, kas ir stāvokļu telpas grafa paveids jeb datu struktūra, kas glabā informāciju par spēles gaitu tās stāvokļu un atļauto gājienu terminos. Heiristiski informētas pārmeklēšanas algoritmi divpersonu spēļu realizācijai realizē pārmeklēšanu, apstrādājot spēles koku, lai dators varētu izpildīt gājienus un virzīt spēli uz priekšu. Heiristiskā novērtējuma funkcija ļauj datoram novērtēt gājiena rezultātā veidojušos spēles stāvokļu izdevību un izvēlēties tādu gājienu, kas visticamāk vedīs uz datoram labvēlīgu iznākumu (uzvaras stāvokli vai sliktākajā gadījumā uz neizšķirtu rezultātu).

Divpersonu spēļu ar pilnu informāciju realizācijas algoritma izvēle ir atkarīga no spēles koka pieejamības. **Pilnībā pieejams (jeb pilns) spēles koks** ir spēles koks, kas atspoguļo visus spēlē atļautos gājienus no spēles sākumstāvokļa, ko atspoguļo spēles koka saknes virsotne, līdz visiem spēles beigu stāvokļiem, ko atspoguļo strupceļa virsotnes. **Daļēji pieejams (jeb daļējais) spēles koks** ir pilna spēles koka apakškoks, kas ir ģenerēts līdz noteiktajam dziļumam. Zemāk ir izskaidroti divpersonu spēļu ar pilnu informāciju realizācijas pamataspekti atkarībā no spēles koka pieejamības.

Izmantošanas gadījumi

Spēles koks var būt pilnībā pieejams vienkāršām spēlēm, kurās stāvokļu skaits ir neliels, vai arī tad, kad ir pieejami pietiekami skaitļošanas resursi, lai spēles koku glabātu un apstrādātu sistēmas darbības laikā. Spēles koks tipiski ir daļēji pieejams, realizējot sarežģītas spēles, kurās stāvokļu skaits ir milzīgi liels, vai arī tad, kad nav pieejami pietiekami skaitļošanas resursi, lai spēles koku glabātu un apstrādātu sistēmas darbības laikā.

Spēles koka izveide

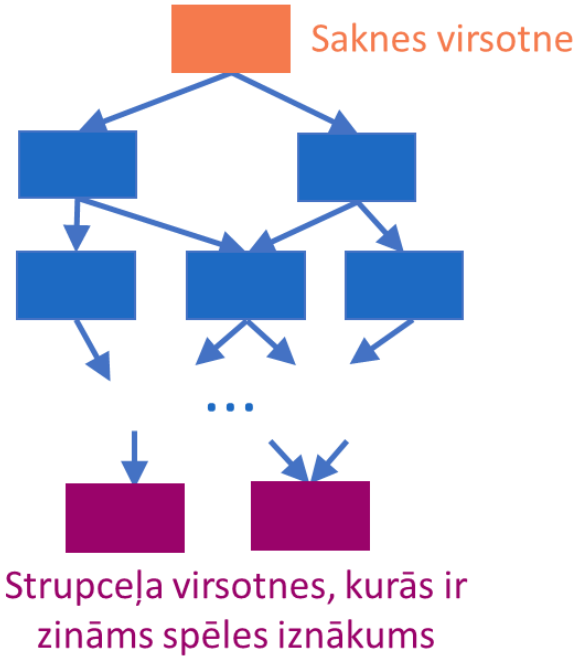
Pilnībā pieejama spēles koka gadījumā spēles koku var:

- izveidot un saglabāt kādā krātuvē (failā, datubāzē, utml.) sistēmas projektēšanas un izstrādes stadijā, vai
- ģenerēt sistēmas darbības laikā, ja ir viegli formalizējami spēles stāvokļu-pāreju likumi.

Daļēji pieejama spēles koka gadījumā spēles koks tiek daļēji ģenerēts, t.i. līdz iepriekš noteiktajam dziļumam, sistēmas darbības laikā.

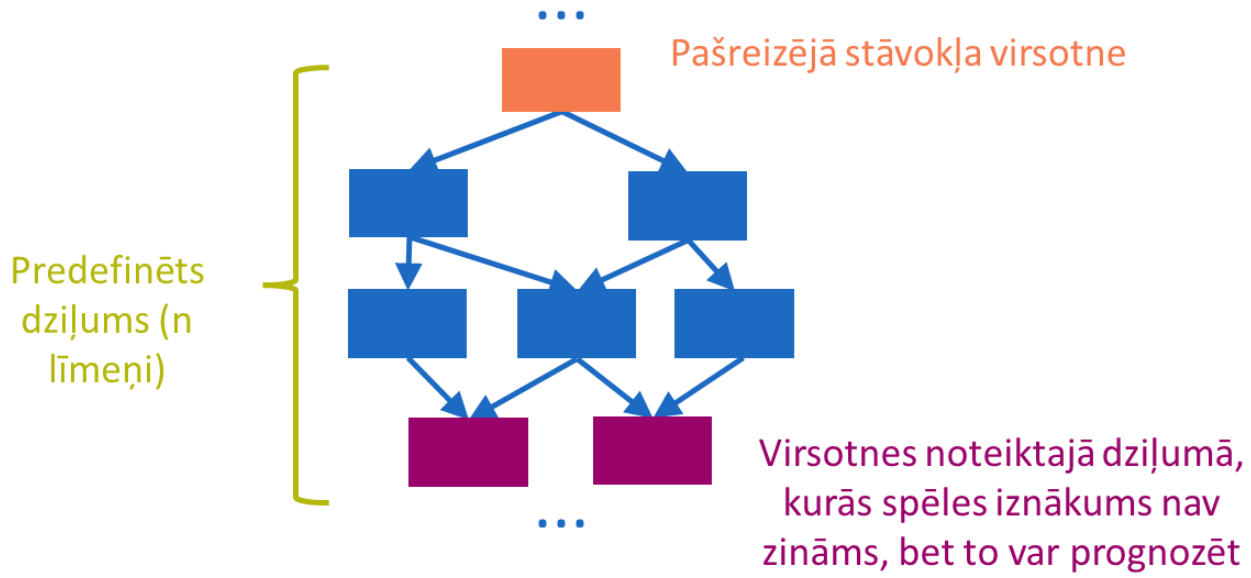
Algoritma pielietojums

Pilnībā pieejama spēles koka gadījumā spēles algoritms tiek pielietots pilnam spēles kokam, kurā ir precīzi zināmas saknes un strupceļa virsotnes, kā arī spēles iznākums katrā no strupceļa virsotnēm (1.attēls).



1. att. Virsotņu pieejamība pilnībā pieejama spēles koka gadījumā

Daļēji pieejama spēles koka gadījumā spēles algoritms tiek pielietots spēles koka apakškokam, kurā saknes un strupceļa virsotnes lielākoties nav zināmas. Šo virsotņu vietā algoritmam ir pieejama spēles pašreizējā stāvokļa virsotne un apakškoka beigu virsotnes noteiktajā dziļumā, kurās nav zināms spēles iznākums (2.attēls).



2. att. Virsotņu pieejamība daļēji pieejama spēles koka gadījumā

Spēlētāju apzīmēšana

Abos gadījumos spēlētājus nosacīti sauc par:

- **maksimizētāju** – šī spēlētāja stratēģija ir darīt visu iespējamo, lai uzvarētu spēli;
- **minimizētāju** - šī spēlētāja nosacītā stratēģija ir darīt visu iespējamo, lai neļautu maksimizētājam uzvarēt, cenšoties novest spēli uz stāvokli, kas ir vissliktākais maksimizētājam.

Spēles koka sadalīšana līmeņos

Abos gadījumos spēles koku/apakškoku pamīšus sadala maksimizētāja (apzīmē MAX) un minimizētāja (apzīmē MIN) līmeņos, atceroties, ka divpersonu spēlēs ar pilnu informāciju gājieni tiek izpildīti secīgi. Spēles koka dalīšana ir patvaļīga, tas nozīmē, ka ir vienalga, ar kuru līmeni to uzsāk. Minimizētāja līmenis norāda, ka no šajā līmenī iespējamajiem spēles stāvokļiem vienu gājienu veic minimizētājs. Maksimizētāja līmenis norāda, ka no šajā līmenī iespējamajiem spēles stāvokļiem vienu gājienu veic maksimizētājs.

Heiristisko vērtējumu piešķiršanas nepieciešamība

Abos gadījumos spēles koka/apakškoka virsotnēm dators piešķir heiristiskos vērtējumus, kas ļauj tam izvēlēties labākos gājienus.

Izmantojamie heiristiskie vērtējumi

Pilnībā pieejama spēles koka gadījumā heiristiskā novērtējuma funkcijas izmantošana nav nepieciešama. Tipiski, ja spēlei ir divi iznākumi (vienmēr viens spēlētājs uzvar, bet otrs zaudē), izmanto šādus vērtējumus:

- 1 – ja virsotne atbilst maksimizētāja uzvarai;
- -1 – ja virsotne atbilst minimizētāja uzvarai;

Ja spēlei ir trīs iznākumi (uzvara, zaudējums un neizšķirts rezultāts), tad tiek izmantoti trīs vērtējumi:

- 1 – ja virsotne atbilst maksimizētāja uzvarai;
- 0 – ja virsotne atbilst neizšķirtam rezultātam;
- -1 – ja virsotne atbilst minimizētāja uzvarai.

Daļēji pieejama spēles koka gadījumā virsotņu heiristiskie vērtējumi tiek iegūti, izmantojot heiristiskā novērtējuma funkciju.

Heiristisko vērtējumu piešķiršanas rezultāts

Pilnībā pieejama spēles koka gadījumā saknes virsotnei piešķirto heiristisko vērtējumu sauc par **spēles rezultātu**, jo tas skaidri parāda, kāds būs spēles iznākums, ja abi spēlētāji spēlēs racionāli, vadot spēli uz uzvaru un nepieļaujot kļūdas.

Daļēji pieejama spēles koka gadījumā **spēles rezultāts nav pieejams**! Pašreizējā spēles stāvokļa virsotnei piešķirtais heiristiskais vērtējums norāda tikai un vienīgi uz labāko spēles stāvokli, ko var sasniegt, izdarot n-gājienus no pašreizējā stāvokļa.

Ceļu identificēšana

Pilnībā pieejama spēles koka gadījumā, pamatojoties uz spēles rezultātu, tiek identificēti uzvaru nesošie ceļi. Tie nav ceļi, kas ved uz viena vai cita spēlētāja uzvaru. Tie ir ceļi, kas norāda, kā spēle norisināsies. Uzvaru nesošs ceļš ir ceļš, kas sākas saknes virsotnē un iet līdz strupceļa virsotnei, apmierinot šādus nosacījumus:

- visas ceļa virsotnes ir savstarpēji saistītas;
- visām ceļa virsotnēm ir vienādi heiristiskie vērtējumi, kas sakrīt ar spēles rezultātu.
- Daļēji pieejama spēles koka gadījumā pēc labākā stāvokļa identificēšanas tiek noteikti ceļi, kas ved uz šo stāvokli. Tie nav uzvaru nesošie ceļi! Tie ir ceļi, kas ved uz labāko stāvokli pēc n-gājieniem.

Izmantojamie algoritmi

Pilnībā pieejama spēles koka gadījumā tiek izmantoti Minimaksa algoritms vai Alfa-beta algoritms. Daļēji pieejama spēles koka gadījumā lieto Pārlūkošana uz priekšu pār n-gājieniem vai Progresējošās iedziļināšanas metodi.

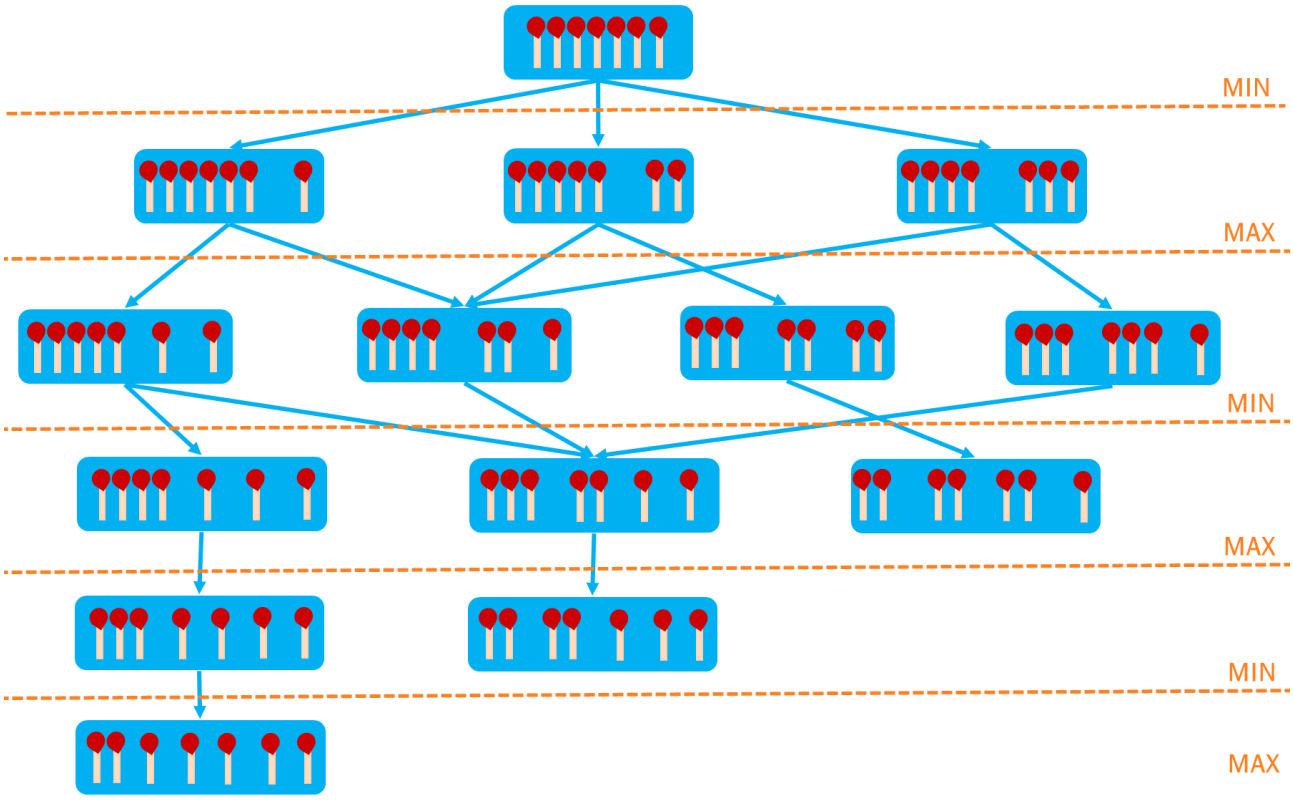
Minimaksa algoritms ietver sevī šādus soļus (Luger, 2009):

1. pilna spēles koka izveide;
2. spēles koka sadalīšana MAX un MIN līmeņos;

3. heiristisko vērtējumu piešķiršana visām strupceļa virsotnēm, analizējot, kurš spēlētājs uzvar konkrētajā virsotnē;
4. strupceļa virsotņu heiristisko vērtējumu pārnešana uz augšu līdz spēles koka saknei, pamatojoties uz šādiem nosacījumiem: a) ja kārtējā spēles koka virsotne atrodas maksimizētāja līmenī, tad virsotnei piešķir maksimālo heiristisko vērtējumu no šīs virsotnes tiešo pēcteču heiristiskajiem vērtējumiem, vai b) ja kārtējā spēles koka virsotne atrodas minimizētāja līmenī, tad virsotnei piešķir minimālo heiristisko vērtējumu no šīs virsotnes tiešo pēcteču heiristiskajiem vērtējumiem;
5. spēles rezultāta un uzvaru nesošu ceļu noteikšana.

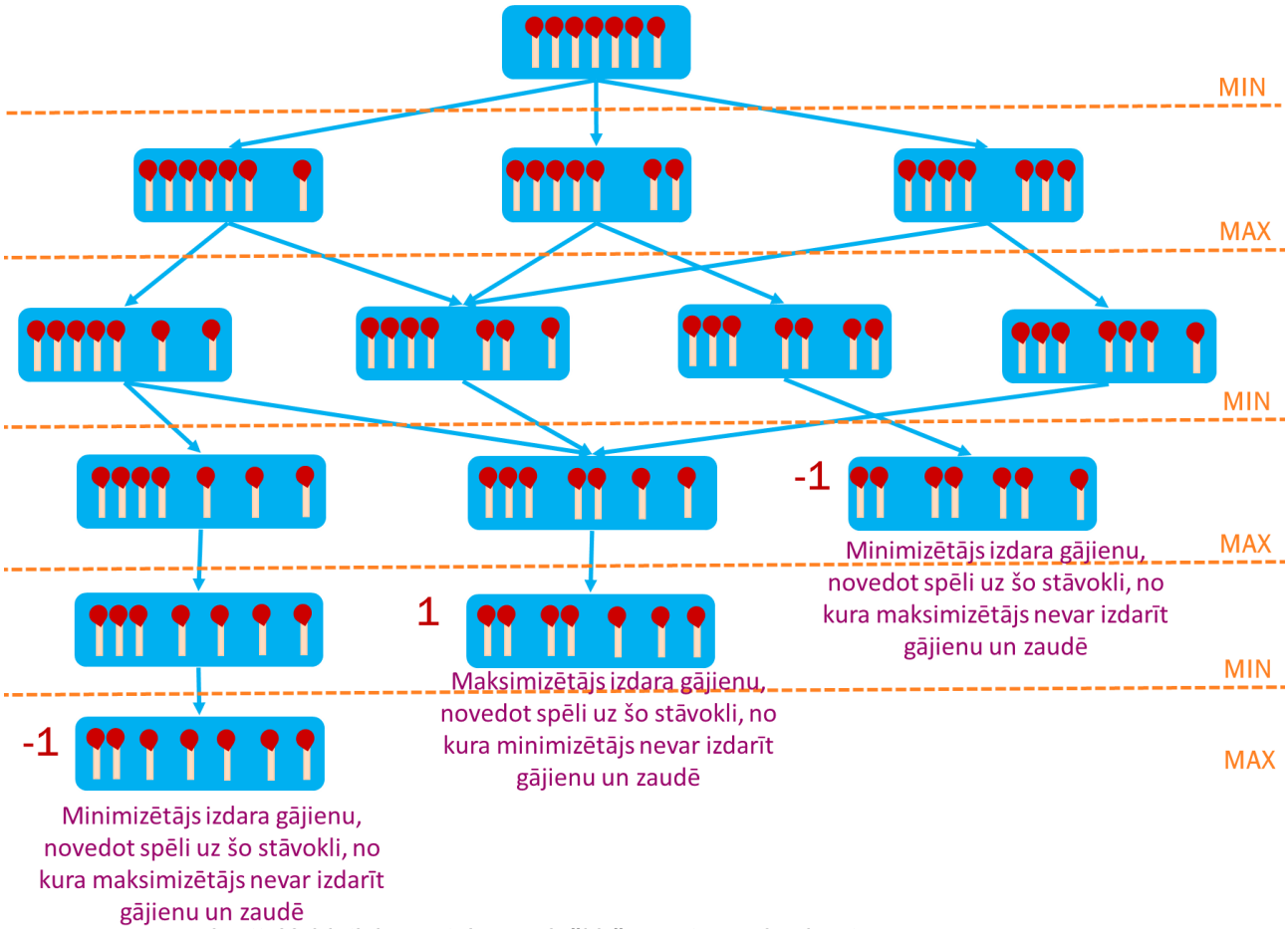
Apskatīsim Minimaksa algoritma darbību, izmantojot par pamatu spēli "Nims". Spēles sākumā ir dota kaudzīte ar septiņiem sērkociņiem. Spēlētāji izpilda gājienus secīgi, katrā gājienā sadalot kaudzīti divās nevienādās daļās. Zaudē spēlētājs, kurš vairs nevar izpildīt gājienu.

Vispirms spēles koks tiek sadalīts MIN un MAX līmeņos (3.attēls). Taču atcerieties, ka dalīšanu var sākt ar jebkuru iezīmi: MAX vai MIN.



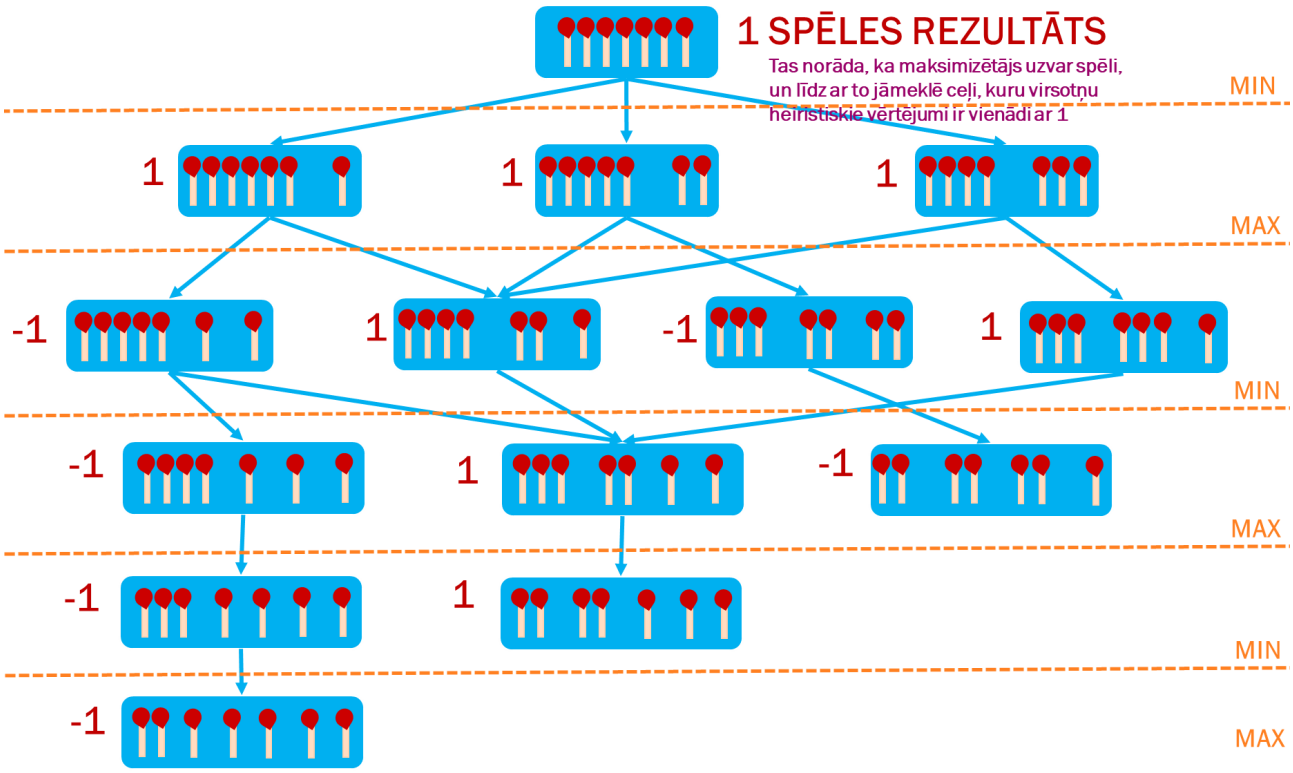
3. att. Spēles koka sadalīšana līmeņos (aizgūts no (Luger, 2009))

Nākamajā solī strupceļa virsotnēm tiek piešķirti heiristiskie vērtējumi (4.attēls).



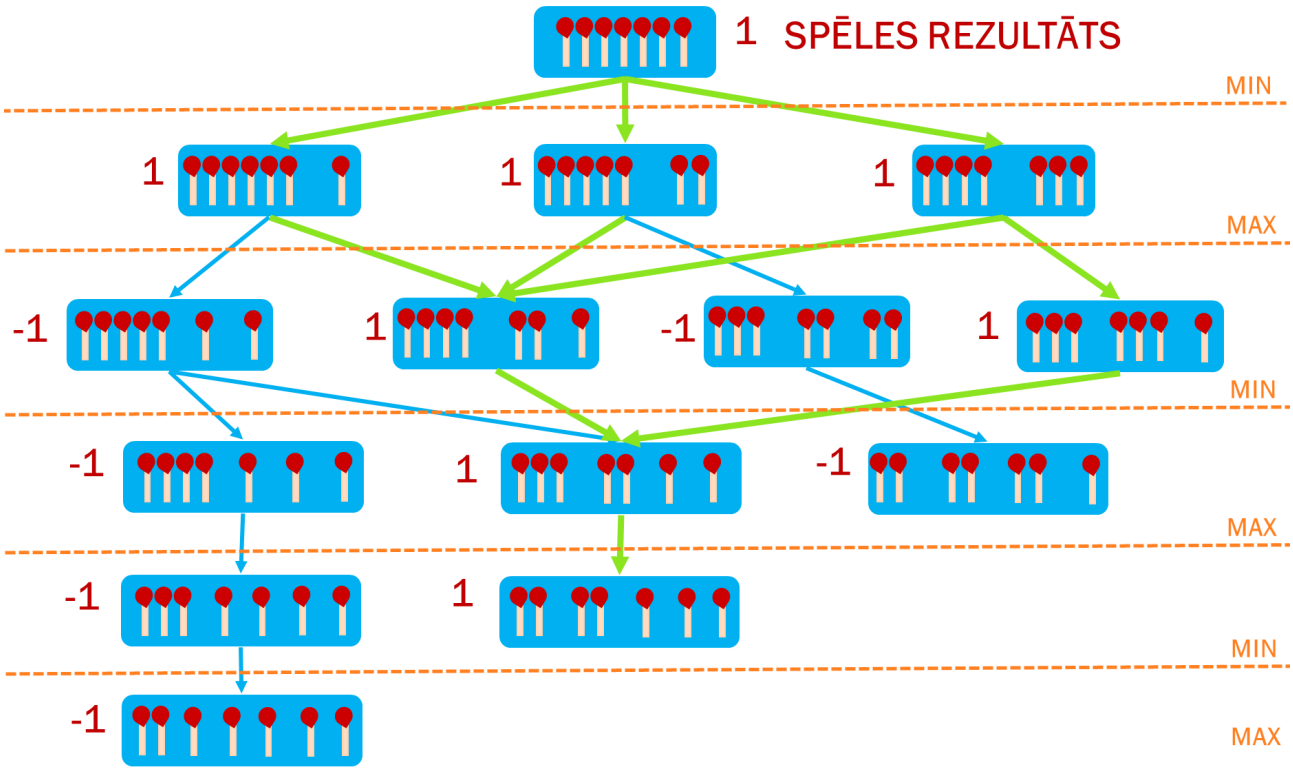
4. att. Heiristisko vērtējumu piešķiršana strupceļa virsotnēm

Kad visas strupceļa virsotnes ir novērtētas, heiristiskie vērtējumi tiek pārnesti uz augšu līdz koka saknei (5.attēls).



5. att. Heiristisko vērtējumu pārnešana uz augšu līdz spēles koka saknei

Pēc vērtējumu pārnešanas var redzēt, ka saknes virsotnei ir piešķirts heiristiskais vērtējums 1. Tas ir spēles rezultāts. Tas norāda, ka maksimizētājs uzvar spēli un līdz ar to ir jāmeklē uzvaru nesošie ceļi, kuru virsotņu heiristiskie vērtējumi ir vienādi ar 1. Kopā ir četri tādi ceļi (6.attēls). Salīdzinot šos ceļus, Jūs redzēsiet, ka visām virsotnēm šajos ceļos heiristiskie vērtējumi ir vienādi ar 1.



6. att. Uzvaru nesošo ceļu identificēšana (modificēts no (Luger, 2009))

Minimaksa algoritms apstrādā visas virsotnes spēles kokā un līdz ar to var prasīt ievērojamus skaitļošanas resursus. **Alfa-beta algoritms** uzlabo pārmeklēšanas efektivitāti. Algoritma pamatā ir ideja par to, ka, ja ir kāds slikts stāvoklis, tad nav jēgas spēlēt, lai pārliecinātos, cik briesmīgi slikts ir šis stāvoklis. Atšķirībā no Minimaksa algoritma šis algoritms nogriež tos spēles koka ceļus, kuri nav perspektīvi, un līdz ar to šim algoritmam nav jāiegūst visu virsotņu heiristiskie vērtējumi. Alfa-beta algoritms uzlabo spēles koka pārmeklēšanas efektivitāti, samazinot novērtējamo virsotņu skaitu un līdz ar to arī kopējo pārmeklēšanas laiku (Luger, 2009).

Alfa-beta algoritms piešķir divus skaitļus (Jones, 2009; Luger, 2009):

- **alfa skaitli** piešķir virsotnei, kura atrodas MAX līmenī. Šī vērtība nedrīkst samazināties. Ja tā samazinās, tad tas ir slikts gājiens. Tādējādi šis skaitlis definē vislabāko gājienu, ko var izdarīt, lai maksimizētu spēles rezultātu;
- **beta skaitli** piešķir virsotnei, kura atrodas MIN līmenī. Šī vērtība nedrīkst palielināties. Ja tā palielinās, tad tas ir slikts gājiens. Tādējādi šis skaitlis definē vislabāko gājienu, ko var izpildīt, lai minimizētu spēles rezultātu.

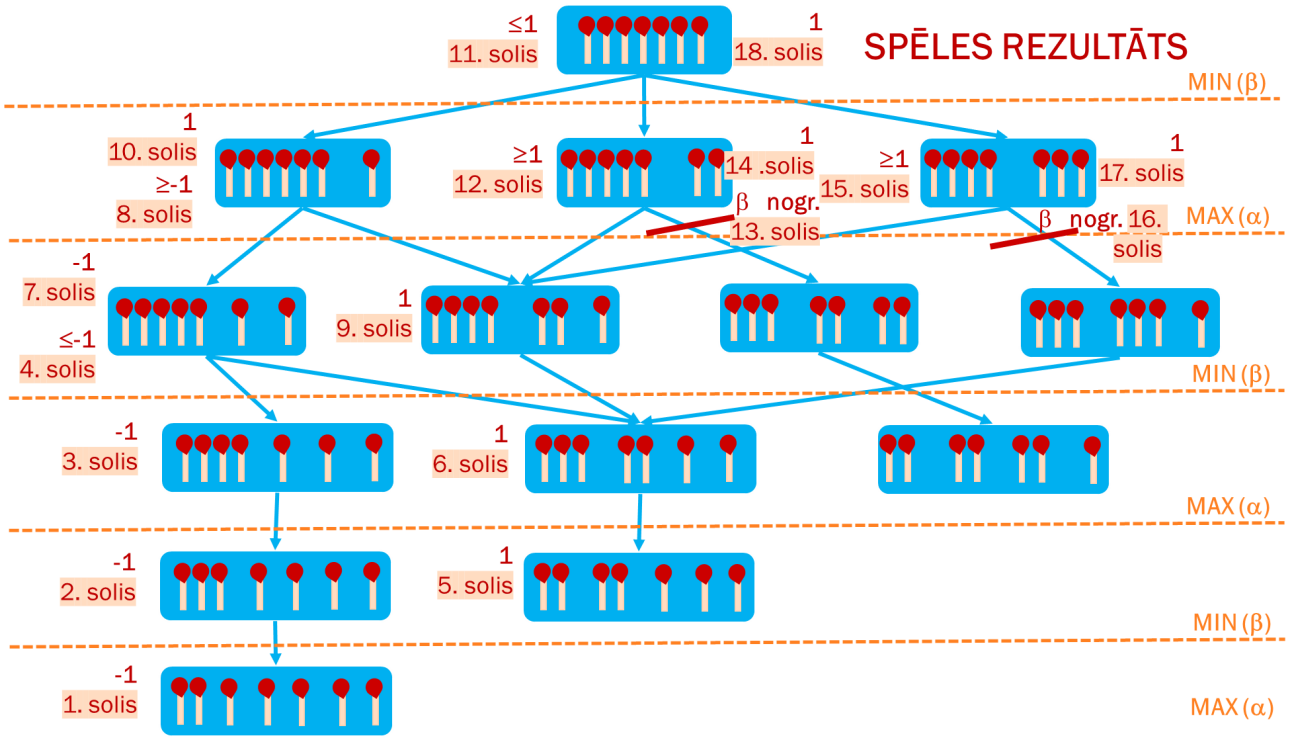
Algoritms izmanto divus likumus, lai veiktu neperspektīvu ceļu nogriešanu, pamatojoties uz alfa un beta vērtībām (Luger, 2009):

- **alfa–nogriešana** - pārmeklēšanu pārtrauc zem jebkuras MIN līmeņa virsotnes, kurai beta vērtība ir mazāka vai vienāda ar alfa vērtību (beta≤alfa) jebkurai no šīs virsotnes tiešo priekšteču virsotnēm;

- **beta–nogriešana** - pārmeklēšanu pārtrauc zem jebkuras MAX līmeņa virsotnes, kurai alfa vērtība ir lielāka vai vienāda ar beta vērtību (alfa≥beta) jebkurai no šīs virsotnes tiešo priekšteču virsotnēm.

Izpildot nogriešanu, ir jāņem vērā, ka tiek nogriezti vēl neapskatīti ceļi.

Apskatīsim algoritma darbību, izmantojot par pamatu spēli "Nims" un jau iepriekš izveidoto spēles koku (3.attēls). Šis algoritms pārmeklē spēles koku pārmeklēšanas dziļumā manierē (Ertel, 2011), t.i. tas sāk ar kreiso pusi un vienmēr nolaižas pēc iespējas dziļāk spēles kokā, vai nu līdz strupceļa virsotnei, vai nu līdz virsotnei, kurai ir piešķirts precīzs heiristisks vērtējums. Šīs tēmas beigās Jums ir pieejams šī algoritma soļu uzskaitījums, lūdzu, izmantojiet to, lai izprastu zemāk doto algoritma realizācijas piemēru (7.attēls).



7. att. Alfa-beta algoritma izpildes piemērs

1.solī algoritms nolaižas pa pirmo vēl neapskatīto ceļu līdz strupceļa virsotnei un piešķir šai virsotnei precīzu heiristisku vērtējumu "-1", jo tajā maksimizētājs zaudē spēli, tāpēc ka pēc minimizētāja gājiena vairs nevar izdarīt nākamo gājienu. Tā kā ir iegūts precīzs heiristisks vērtējums, algoritms uzreiz dodas atpakaļ pa to pašu ceļu, pa kuru bija nolaidis, ar mērķi pārnest iegūto heiristisko vērtējumu uz augšu, un 2. un 3. solī algoritms piešķir precīzus heiristiskus vērtējumus, kas ir vienādi ar strupceļa virsotnes vērtējumu, jo apskatāmajām virsotnēm nav citu pēcteču.

4.solī, turpinot heiristiskā vērtējuma pārnešanu, algoritms konstatē, ka virsotnei "5-1-1" ir vēl viens neapskatīts pēctecis. Šajā gadījumā nav iespējams piešķirt precīzu heiristisku vērtējumu un algoritms piešķir starpvērtējumu "<=-1", ņemot vērā līmeni, kurā virsotne ir izvietota, un vienīgā līdz šim atrastā pēcteča heiristisko vērtējumu. Piešķirtais starpvērtējums norāda, ka šajā virsotnē algoritms sliktākajā gadījumā paņems vērtējumu "-1", bet labākajā gadījumā mazāko vērtējumu, jo minimizētāja uzdevums ir vienmēr minimizēt iznākumu.

Tā kā ir iegūts starpvērtējums, algoritms pārbauda, vai virsotnes “5-1-1” priekštecim arī ir starpvērtējums. Šī pārbaude dod negatīvu rezultātu, tāpēc algoritms uzreiz nolaižas pa vēl neapskatīto ceļu līdz strupceļa virsotnei un 5.solī iegūst tās heiristisko vērtējumu “1”, jo šis stāvoklis veidojas pēc maksimizētāja gājiena un minimizētājam nav iespēju izpildīt nākamo gājienu, līdz ar to viņš zaudē spēli.

Tā kā ir iegūts precīzs heiristisks vērtējums, algoritms uzreiz dodas atpakaļ pa to pašu ceļu, pa kuru bija nolaidis, ar mērķi pārnest iegūto heiristisko vērtējumu uz augšu, un 6.solī piešķir precīzu heiristisku vērtējumu, kas ir vienāds ar strupceļa virsotnes vērtējumu, jo apskatāmajai virsotnei nav citu pēcteču.

Tā kā ir iegūts precīzs heiristisks vērtējums, algoritms uzreiz dodas atpakaļ pa to pašu ceļu, pa kuru bija nolaidis, ar mērķi pārnest iegūto heiristisko vērtējumu uz augšu, un 7.solī algoritms piešķir precīzu heiristisku vērtējumu “-1”, jo šai virsotnei vairs nav neapskatīto pēcteču. Turklāt, vērtējums, kas ir mazāks par “-1”, netika atrasts, tāpēc, ņemot vērā, ka virsotne atrodas MIN līmenī, algoritms paņem mazāko vērtējumu no atrastajiem pēcteču vērtējumiem.

Tā kā ir iegūts precīzs heiristisks vērtējums, algoritms uzreiz dodas atpakaļ pa to pašu ceļu, pa kuru bija nolaidis, ar mērķi pārnest iegūto heiristisko vērtējumu uz augšu, un konstatē, ka virsotnei “6-1” ir vēl viens neapskatīts pēctecis. Šajā gadījumā nav iespējams piešķirt precīzu heiristisku vērtējumu, un algoritms 8.solī piešķir starpvērtējumu “>= -1”, ņemot vērā līmeni, kurā virsotne ir izvietota un vienīgā līdz šim atrastā pēcteča heiristisko vērtējumu. Piešķirtais starpvērtējums norāda, ka šajā virsotnē algoritms sliktākajā gadījumā paņems vērtējumu “-1”, bet labākajā gadījumā lielāko vērtējumu, jo maksimizētāja uzdevums ir vienmēr maksimizēt iznākumu.

Tā kā ir iegūts starpvērtējums, algoritms pārbauda, vai virsotnes “6-1” priekštecim arī ir starpvērtējums. Šī pārbaude dod negatīvu rezultātu, tāpēc algoritms uzreiz nolaižas pa vēl neapskatīto ceļu līdz virsotnei “3-2-1-1”, jo šai virsotnei jau ir iegūts precīzs heiristisks vērtējums, un 9.solī pārnes tās heiristisko vērtējumu “1” uz augšu, piešķirot precīzu heiristisku vērtējumu, jo virsotnei šajā līmenī vairs nav pēcteču.

Tā kā ir iegūts precīzs heiristisks vērtējums, algoritms uzreiz dodas atpakaļ pa to pašu ceļu, pa kuru bija nolaidis, ar mērķi pārnest iegūto heiristisko vērtējumu uz augšu, un 10.solī algoritms piešķir precīzu heiristisku vērtējumu “1”, jo šai virsotnei vairs nav neapskatīto pēcteču. Turklāt, vērtējums, kas ir lielāks par “1”, netika atrasts, tāpēc, ņemot vērā, ka virsotne atrodas MAX līmenī, algoritms paņem lielāko vērtējumu no atrastajiem pēcteču vērtējumiem.

Tā kā ir iegūts precīzs heiristisks vērtējums, algoritms uzreiz dodas atpakaļ pa to pašu ceļu, pa kuru bija nolaidis, ar mērķi pārnest iegūto heiristisko vērtējumu uz augšu, un konstatē, ka saknes virsotnei ir vēl neapskatīti pēcteči. Šajā gadījumā nav iespējams piešķirt precīzu heiristisku vērtējumu, un algoritms 11.solī piešķir starpvērtējumu “<=1”, ņemot vērā līmeni, kurā virsotne ir izvietota un vienīgā līdz šim atrastā pēcteča heiristisko vērtējumu. Piešķirtais starpvērtējums norāda, ka šajā virsotnē algoritms sliktākajā gadījumā paņems vērtējumu “1”, bet labākajā gadījumā mazāko vērtējumu, jo minimizētāja uzdevums ir vienmēr minimizēt iznākumu.

Tā kā ir iegūts starpvērtējums un saknes virsotnei nav priekšteča, algoritms uzreiz nolaižas pa vēl neapskatīto ceļu līdz virsotnei “4-2-1”, jo šai virsotnei jau ir iegūts precīzs heiristisks vērtējums, un pārnes tās heiristisko vērtējumu uz augšu. Pārnešanas laikā algoritms konstatē, ka virsotnei šajā līmenī ir vēl neapskatīts pēctecis. Šajā gadījumā nav iespējams piešķirt precīzu heiristisku vērtējumu, un algoritms 12.solī piešķir starpvērtējumu “>=1”, ņemot vērā līmeni, kurā virsotne ir

izvietota, un vienīgā līdz šim atrastā pēcteča heiristisko vērtējumu. Piešķirtais starpvērtējums norāda, ka šajā virsotnē algoritms sliktākajā gadījumā paņems vērtējumu “1”, bet labākajā gadījumā lielāko vērtējumu, jo maksimizētāja uzdevums ir vienmēr maksimizēt iznākumu.

Pēc starpvērtējuma piešķiršanas 12.solī, veidojas situācija, kad ir pieejami divi starpvērtējumi virsotnēm, kas veido ceļu, pa kuru algoritms ir nolaidis: virsotnei “5-2” ir starpvērtējums “>=1” un saknes virsotnei – “<=1”. Šādā situācijā tiek pārbaudīta konflikta esamība starp diviem starpvērtējumiem. Var redzēt, ka abām virsotnēm ir kopīga tikai viena vērtība “1” jeb, citiem vārdiem sakot, virsotnes “5-2” alfa-skaitlis ir vienāds ar saknes virsotnes beta- skaitli un tas nozīmē to, ka ir jāveic beta-nogriešana, nogriežot zem virsotnes “5-2” vēl neapskatītos ceļus. Tādējādi 13.solī tiek veikta beta-nogriešana.

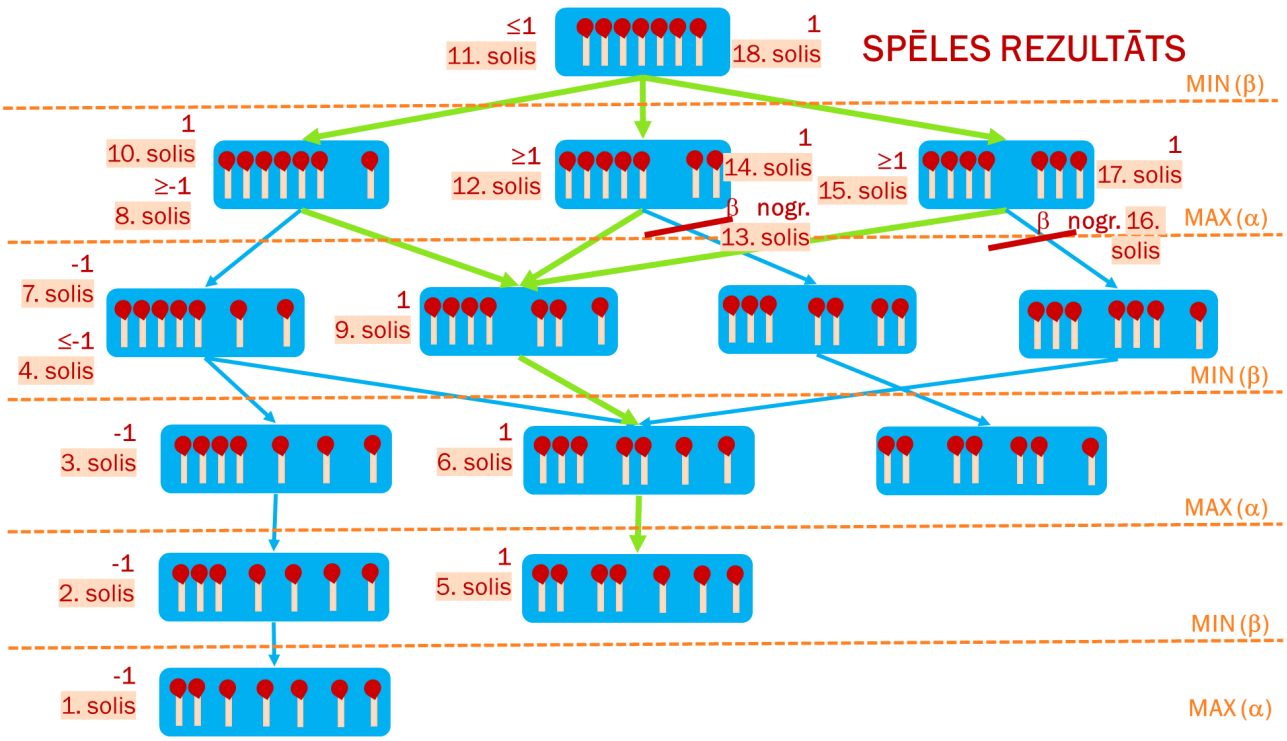
Pēc nogriešanas izpildes, virsotnei, zem kuras notika nogriešana, tiek piešķirts precīzs heiristisks vērtējums, pamatojoties uz vienīgā novērtētā pēcteča heiristisko vērtējumu. Tādējādi 14.solī virsotne “5-2” iegūst precīzu heiristisku vērtējumu “1”.

Tā kā ir iegūts precīzs heiristisks vērtējums, algoritms uzreiz dodas atpakaļ pa to pašu ceļu, pa kuru bija nolaidis, ar mērķi pārnest iegūto heiristisko vērtējumu uz augšu. Sasniedzot saknes virsotni, tas konstatē, ka nav iespējams mainīt tās heiristisko vērtējumu, jo ir iegūts tas pats vērtējums “1”. Līdz ar to algoritms nolaižas pa vēl neapskatīto ceļu līdz virsotnei “4-2-1”, jo šai virsotnei jau ir iegūts precīzs heiristisks vērtējums, un pārnes tās heiristisko vērtējumu uz augšu. Pārnešanas laikā algoritms konstatē, ka virsotnei “4-3” ir vēl neapskatīts pēctecis. Šajā gadījumā nav iespējams piešķirt precīzu heiristisku vērtējumu, un algoritms 15.solī piešķir starpvērtējumu “>=1”, ņemot vērā līmeni, kurā virsotne ir izvietota, un vienīgā līdz šim atrastā pēcteča heiristisko vērtējumu. Piešķirtais starpvērtējums norāda, ka šajā virsotnē algoritms sliktākajā gadījumā paņems vērtējumu “1”, bet labākajā gadījumā lielāko vērtējumu, jo maksimizētāja uzdevums ir vienmēr maksimizēt iznākumu.

Pēc starpvērtējuma piešķiršanas 15.solī, veidojas situācija, kad ir pieejami divi starpvērtējumi virsotnēm, kas veido ceļu, pa kuru algoritms ir nolaidis: virsotnei “4-3” ir starpvērtējums “>=1” un saknes virsotnei – “<=1”. Šādā situācijā tiek pārbaudīta konflikta esamība starp diviem starpvērtējumiem. Var redzēt, ka abām virsotnēm ir kopīga tikai viena vērtība “1” jeb, citiem vārdiem sakot, virsotnes “4-3” alfa-skaitlis ir vienāds ar saknes virsotnes beta-skaitli un tas nozīmē to, ka ir jāveic beta-nogriešana, nogriežot zem virsotnes “4-3” vēl neapskatītos ceļus. Tādējādi 16.solī tiek veikta beta-nogriešana.

Pēc nogriešanas izpildes, virsotnei, zem kuras notika nogriešana, tiek piešķirts precīzs heiristisks vērtējums, pamatojoties uz vienīgā novērtētā pēcteča heiristisko vērtējumu. Tādējādi 17.solī virsotne “4-3” iegūst precīzu heiristisku vērtējumu “1”.

Tā kā ir iegūts precīzs heiristisks vērtējums, algoritms uzreiz dodas atpakaļ pa to pašu ceļu, pa kuru bija nolaidis, ar mērķi pārnest iegūto heiristisko vērtējumu uz augšu. Sasniedzot saknes virsotni, tas konstatē, ka tai vairs nav neapskatīto pēcteču, tāpēc 18.solī piešķir precīzu heiristisku vērtējumu jeb spēles rezultātu, kas ir vienāds ar “1”, jo visu pēcteču heiristiskie vērtējumi ir vienādi ar “1” un nav iespējams paņemt mazāko vērtējumu. Tādējādi spēlējot racionāli, jebkurā gadījumā uzvarēs maksimizētājs. Pamatojoties uz spēles rezultātu, tiek noteikti uzvaru nesošie ceļi (8.attēls).



8.att. Uzvaru nesošo ceļu noteikšana Alfa-beta algoritmā

Pārlūkošana uz priekšu pār *n*-gājieniem var tikt balstīta gan uz Minimaksa, gan uz Alfa-beta algoritmu principiem. Tās darbība ne ar ko neatšķiras no minētajiem algoritmiem, izņemot to, ka algoritms apstrādā noteiktā dziļuma spēles koka apakškoku un ir jāizmanto heiristiskā novērtējuma funkcija virsotņu kvalitātes novērtēšanai. Viena no lielākajām problēmām šajā pieejā ir nosaukta par **horizonta efektu**. Dziļums, līdz kuram tiek pētīts spēles koks, var nenoteikt, ka ceļš, kurš visu laiku ir bijis heiristiski daudzsološs, beigās var novest sliktā situācijā, kuras rezultātā tiks zaudēta visa spēle. Papildus problēma ir saistīta ar **nestabiliem stāvokļiem**, t.i. tādiem stāvokļiem, kuru heiristiskie vērtējumi krasi atšķiras, izpildot dažādus gājienus no viena un tā paša spēles stāvokļa. Minēto problēmu galvenais iemesls ir ne visai labi informēta heiristiskā novērtējuma funkcija. (šis fragments ir veidots, pamatojoties uz informāciju, kas ir dota (Luger, 2009))

Progresējošās iedziļināšanas metode tiek izmantota spēlēm, kurās ir laika ierobežojums gājiena izpildei. Šādās spēlēs nav iespējams izmantot Pārlūkošanu uz priekšu pār n-gājieniem, jo saģenerētais apakškoks var būt tik liels, ka gājienam atvēlētais laiks var iztecēt, kamēr tiek veikta apakškoka novērtēšana. Progresējošās iedziļināšanas metode arī var tikt balstīta gan uz Minimaksa, gan uz Alfa-beta algoritmu principiem. Tā novērtē apakškoku pakāpeniski, pa līmeņiem. Tā uzsāk novērtēšanu ar 1.līmeņa virsotnēm, tās novērtē, izmantojot heiristiskā novērtējuma funkciju, un tad heiristiskus vērtējumus pārnes uz augšu, pamatojoties uz Minimaksa vai Alfa-beta algoritma principiem. Novērtēšanas rezultātus saglabā. Ja vēl ir pieejams laiks, tad pāriet pie nākamā līmeņa virsotnēm un izdara visu to pašu, vienlaikus sekojot tam, cik daudz laika ir atlicis. Tātad apakškoks tiek pētīts līmeni pa līmenim un, ja gājienam atvēlētais laiks tuvojas beigām, algoritms pārtrauc darbību un izmanto iepriekšējā līmeņa novērtēšanas rezultātus, lai izpildītu kārtējo gājienu. (šis fragments ir veidots, pamatojoties uz informāciju, kas ir dota (Luger, 2009))